

Implementación de una plataforma de software móvil Android para la gestión de servicios informativos y de asistencia técnica para la reparación y el mantenimiento de motocicletas en el municipio de Rionegro.

Autor

Adolfo Esteban Agudelo Flórez.

UNIVERSIDAD CATÓLICA DE ORIENTE

FACULTAD DE INGENIERÍAS

PROGRAMA DE INGENIERÍA DE SISTEMAS

RIONEGRO

2026

Implementación de una plataforma de software móvil Android para la
gestión de servicios informativos y de asistencia técnica para la
reparación y el mantenimiento de motocicletas en el municipio de
Rionegro

Autor

Adolfo Esteban Agudelo Flórez.

Informe trabajo de grado.

Asesor

Wider Farid Sánchez Garzón

UNIVERSIDAD CATÓLICA DE ORIENTE

FACULTAD DE INGENIERÍAS

PROGRAMA DE INGENIERÍA DE SISTEMAS

RIONEGRO

2026

Notas de aceptación:

Firma del presidente del jurado

Firma del jurado

Firma del jurado

Rionegro, viernes 20 de marzo del 2026

Dedicatoria:

A Ruth Bernarda Gómez Valencia, por sus consejos siempre llenos de sabiduría y por todo lo que me preparó para cuando llegara este momento.

A José Fernando Londoño Álvarez, por ser una persona siempre servicial y atenta, permitiéndome aprovechar el tiempo al máximo.

A Emmanuel Londoño Gómez, por todo lo que compartimos desde el inicio de la carrera hasta ahora y por las largas noches de trabajó en equipo.

A José David Londoño Gómez, por su ejemplo y por brindar espacios de esparcimiento, cortos pero de mucho provecho para continuar.

A Valentina Escobar Bedoya, por su apoyo incondicional y compañía durante los meses más decisivos de este proceso.

A Nicolas Agudelo Flórez, por estar ahí cuando más lo necesité, ayudándome con la documentación del proyecto y cuidándome en los momentos en que la salud no me acompañó.

Agradecimientos:

A Andrés Henao Osorio, por el tiempo que compartió su experiencia y conocimientos.

A Wider Farid Sánchez Garzón, por su invaluable tiempo, acompañando todo el proceso, enseñando y guiando cada etapa del camino.

A Dios, por haberme permitido terminar este proceso y conocer **a las personas que formaron parte de él.**

Resumen

Los motociclistas tienen un problema por falta de un canal centralizado que les permita buscar servicios técnicos y de insumos confiables en talleres y tiendas de motocicletas, aspectos como conocer la disponibilidad de los servicios o hacerle seguimiento a los que se le realizan a sus vehículos. No son claramente cubiertos, por lo que la búsqueda termina siendo lenta, y muchas veces, imprecisa.

Para abordar la problemática desde la implementación de MotoGo, se trabajó en una plataforma de software móvil Android orientada a conectar los motociclistas con establecimientos de servicio técnico. El desarrollo inició desde el diseño metodológico donde se trabajó la ingeniería de requisitos basada en 88 historias de usuario, con esto se elaboraron los artefactos técnicos donde se definen los diseños y tecnologías de la solución, en la última fase se documenta la construcción y despliegue de la plataforma la cual cuenta con descubrimiento de talleres mediante un mapa interactivo usando filtros por marca, cilindrada y cercanía según la ubicación del usuario, gestión de motocicletas, solicitud de diagnósticos, historial de procesos realizados y administración de sedes con el detalle de los horarios, servicios y calificaciones. En el apartado de resultados se muestra que la solución planteada resuelve la falta de información, reduce tiempos de búsqueda y aporta trazabilidad al ciclo de vida del servicio de motocicletas.

Palabras clave: plataforma móvil, motocicletas, gestión de servicios, asistencia técnica.

Abstract

Motorcyclists have a problem because there is no centralized information about services in order to search for reliable technical and supply services at workshops and motorcycle stores and know the availability of those services or to keep track of the ones that are being used on their vehicles. That search ends up being slow and many times inaccurate. This undergraduate project addresses the problem through the implementation of MotoGo, an Android mobile software platform, thought out to connect motorcyclists with technical service establishments. The development starts from the methodological design where requirements engineering was worked on based on 88 user stories, taking this into account, the work moves on to technical artifacts where the designs and technologies were defined in the last phase, the construction and deployment process of the platform is documented which features workshop discovery through an interactive map using filters by brand, engine displacement and proximity according to the user's location, motorcycle management, diagnostic requests with photographic evidence, history of performed services and ratings, branch administration with schedules and services. The results about of the work show that the platform resolves the lack of information, reduces search times and brings traceability to the service lifecycle of motorcycles.

Keywords: mobile platform, motorcycles, service management, technical assistance, Android.

CONTENIDO

1. ANTECEDENTES	12
2. PLANTEAMIENTO DEL PROBLEMA	15
3. JUSTIFICACIÓN	17
4. OBJETIVOS	19
4.1. General	19
4.2. Específicos	19
5. MARCO TEÓRICO	20
5.1 Gestión de servicios de mantenimiento y reparación	20
5.2 Información confiable y toma de decisiones del usuario	21
5.3 Calidad del servicio y percepción del usuario	22
5.4 Experiencia del motociclista en la búsqueda de servicios	22
5.5 Transparencia y confianza en la prestación de servicios	23
5.6 Tiempo del motociclista	23
5.7 Geolocalización y acceso móvil en la búsqueda de servicios	24
5.8 Reseñas y calificación del servicio	25
6. FASE 1. DISEÑO METODOLÓGICO	25
6.1 Diseño Estratégico — El Qué	26
6.1.2 Visión ¿Hasta dónde vamos a llegar?	26
6.1.3 Mapa de Impacto	27
6.1.4 Modelo de Dominio	28
6.1.4.1 Modelo enriquecido	29
6.1.5 Event Storming	29
6.1.6 Dimensionamiento del Producto	31
6.1.7 Mapa de Historias de Usuario	31
6.1.8 Product Backlog	33
6.1.9 Escritura de Historias de Usuario	33
6.1.10 Plan de Liberación	34
7. FASE 2. ARTEFACTOS TÉCNICOS	35
7.1 Diseño arquitectónico	35
7.1.1 Características de Calidad	36
7.1.1.2 Escenarios de Calidad	37
7.1.1.3 Funcionalidades Esenciales	37
7.1.1.4 Restricciones técnicas	38
7.1.1.5 Restricciones de negocio	38
7.1.2 Spikes	39
7.1.3 Alternativas de solución	39
7.1.4 Arquetipo de solución	40
7.1.5 Plataforma tecnológica	40
7.2 Diseño detallado	41
7.2.1 Diagrama de calses	41

	9
7.2.2 Diagrama de componentes	42
7.2.3 Diagrama de paquete	43
7.2.4 Diagrama de secuencia	43
7.2.5 Diagrama de despliegue	44
7.2.6 Modelo entidad relación	45
8. FASE 3: CONSTRUCCIÓN	46
8.1 Análisis estático de código	46
8.1.2 Pruebas de carga	47
8.1.3 Patrones de diseño	48
8.1.4 Manual de instalación	49
8.2 Devops	49
9. RESULTADOS	50
9.1 Perspectiva del Motociclista	50
9.1.1 Bienvenida y Registro	51
9.1.2 Autenticación	53
9.1.3 Descubrimiento de Sedes en el Mapa	54
9.1.4 Detalle de Sede	55
9.1.5 Gestión de Motocicletas	57
9.1.6 Historial de Servicios y Calificaciones	59
9.1.7 Perfil y Configuración	62
9.1.8 Solicitud de Diagnóstico	64
9.2 Perspectiva del Administrador	66
9.2.1 Catálogos Técnicos	67
9.2.2 Líneas por Marca	68
9.2.3 Categorías de Motocicletas	69
9.2.4 Catálogo de Servicios	70
9.3 Perspectiva del Representante de Sede	71
9.3.1 Mis Sedes	71
9.3.2 Perfil de la Sede	72
9.3.3 Creación de Nueva Sede	76
9.3.4 Gestión de Servicios	77
9.3.5 Búsqueda de Motocicleta por Placa	79
9.3.6 Registro y Seguimiento de Servicios	81
9.3.7 Gestión de Franquicias	83
9.4 Respuesta a la Pregunta de Investigación	84
10. CONCLUSIONES	85
11. REFERENCIAS BIBLIOGRÁFICAS	90
12. ANEXOS	92
13. GLOSARIO	97

Tabla 1*Figuras presentadas en la sección de resultados del proyecto MotoGo*

N° Figura	Nombre
Figura 1	Pantalla de bienvenida con selección de rol
Figura 2	Formulario de registro para motociclista
Figura 3	Formulario de registro de representante de sede
Figura 4	Pantalla de inicio de sesión
Figura 5	Pantalla de recuperación de contraseña
Figura 6	Pantalla con mapa interactivo de descubrimiento de sedes con filtros por marca, cilindraje y tipo de establecimiento
Figura 7	Detalle de sede con información general, servicios ofrecidos y horarios de atención
Figura 8	Sección de reseñas y valoraciones de otros motociclistas
Figura 9	Formulario de registro de una nueva motocicleta
Figura 10	Lista de motocicletas registradas en el perfil del usuario
Figura 11	Formulario de edición de datos de una motocicleta
Figura 12	Historial de servicios realizados a una motocicleta
Figura 13	Detalle de un servicio marcado como finalizado, con información de costos
Figura 14	Sistema de calificación con estrellas y campo de comentario
Figura 15	Pantalla de perfil del usuario con sus datos personales
Figura 16	Pantalla de cambio de contraseña
Figura 17	Menú lateral de navegación de la aplicación
Figura 18	Diálogo de confirmación para cierre de sesión
Figura 19	Diálogo de confirmación para eliminación de cuenta
Figura 20	Formulario de solicitud de diagnóstico con selección de motocicleta, descripción del problema, evidencia fotográfica y vista previa del mensaje
Figura 21	Panel de Administración con accesos a Catálogo de Servicios y Catálogos Técnicos
Figura 22	Menú principal de Catálogos Técnicos con los módulos disponibles
Figura 23	Lista de marcas de motocicletas disponibles en el catálogo
Figura 24	Líneas de motocicleta asociadas a la marca
Figura 25	Lista de categorías de motocicletas con cantidad de líneas por categoría
Figura 26	Líneas de motocicleta dentro de la categoría Adventure
Figura 27	Catálogo de servicios con buscador, filtro por tipo y estado de cada servicio

Figura 28	Formulario de edición de un servicio con nombre, descripción, tipo y toggle de activación
Figura 29	Lista de sedes del representante con buscador y etiquetas de franquicia
Figura 30	Gestión de horarios de atención con vigencia y días de operación
Figura 31	Excepciones de horario con indicador de estado vencido
Figura 32	Ubicación de la sede
Figura 33	Formulario de edición de la información de una sede
Figura 34	Diálogo de confirmación de seguridad para eliminación de sede
Figura 35	Formulario de creación de una nueva sede
Figura 36	Gestión de servicios con toggles de activación y desactivación por sede
Figura 37	Detalle de reseñas con calificación promedio, distribución de estrellas y comentarios individuales
Figura 38	Búsqueda de motocicleta por placa con información completa, evidencias fotográficas y diagnósticos
Figura 39	Formulario de registro de un nuevo servicio con cotización y precio final
Figura 40	Lista de servicios realizados con estados, precios y fechas
Figura 41	Detalle de servicio con historial de cambios tipo timeline
Figura 42	Creación de franquicia con nombre, descripción y asociación de sedes
Figura 43	Detalle de franquicia con sedes asociadas y sedes disponibles

1. ANTECEDENTES

La motocicleta en Colombia es un medio de transporte visto como una herramienta de trabajo y de vida cotidiana para la mayoría de usuarios. con lo que se vió en el informe del Registro Único Nacional de Tránsito (RUNT), el país cerró el año del 2025 con un total de 21.345.925 vehículos activos, los cuales 13.528.164 corresponden a motocicletas, lo que equivale a un 63.36% del parque automotor nacional (El Colombiano, 2026). Las cifras muestran que las motos no solo lideran la composición vehicular, sino que sumado a eso siguen en aumento cada año.

En el momento los mecanismos que hay para la búsqueda de asistencia técnica para la reparación y el mantenimiento de motocicletas puntualmente no está implementada como tal, entonces sucede que se puede buscar servicios generales y estos junto con la información en su mayoría están desactualizados, puede llegar al caso que el establecimiento ya no exista.

Se gasta tiempo en la búsqueda del servicio dependiendo del tipo de servicio y del establecimiento que lo presta porque no están los repuestos o no hay personal capacitado en el momento para alguna motocicleta en específico.

La duda de la calidad del servicio es una constante, porque posiblemente el mecánico de confianza por alguna razón no está disponible, o se está en otra ciudad o municipio y no se puede esperar más tiempo para recibir algún tipo de servicio para la motocicleta.

Los costos que se tienen adicionales por un diagnóstico o un trabajo mal echo, en ocasiones por terceros, donde se le dice al cliente que el establecimiento presta el servicio, el cual no termina siendo cierto, sino que lo presta otro establecimiento, pero cuando el trabajo queda mal hecho, los establecimientos no responden porque se echan la culpa entre ellos.

Adicionalmente hay perdida de tiempo cuando se lleva la motocicleta a un servicio y mandan al cliente a buscar los repuestos, esto sucede porque mientras el personal del establecimiento está ocupado y sabe que no tiene los insumos necesarios entonces le pide al cliente que vaya y haga la búsqueda por cuenta propia para poder brindarle el servicio.

Para darle solución a esa necesidad, hoy existen algunas herramientas, pero ninguna está diseñada específicamente para conectar motociclistas con talleres especializados:

AutoSoft Taller: Plataforma que ofrece una amplia gama de funciones para gestionar la información relacionada con vehículos y clientes en talleres de latonería, pintura y mecánica general. La versión profesional brinda mayor control para la gestión de compañías aseguradoras también cuenta con manejo de insumos y costos, asignación de trabajos al personal técnico un seguimiento de pagos anticipados y registro fotográfico de los procesos antes y después (AutoSoft Taller, s.f.).

Google Maps: Un servicio de mapas y navegación en línea que utiliza este sistema para que los usuarios, así como las empresas, puedan llegar a cualquier lugar. Además, lejos de simplemente proporcionar orientación e información para localizar ofertas de negocios locales, Google Maps es una de esas plataformas donde puedes encontrar reseñas de tiendas. Los usuarios pueden buscar casi cualquier tipo de negocio o servicio en su área a través de la interfaz (en la web y en aplicaciones

móviles), incluyendo restaurantes, tiendas, talleres de reparación de autos y motocicletas, entre otros (Google LLC, s.f.).

ServitechApp: Esta aplicación, disponible para plataformas como Windows, Mac, iPad y iPhone, está dirigida a talleres en general. Sus funcionalidades incluyen la creación de órdenes y cotizaciones, gestión de inventario, verificación rápida del estado de las reparaciones de los vehículos en el taller, registro de mantenimientos y notificación a los clientes sobre próximas visitas. Además, es personalizable con el logo de la empresa (ServitechApp, s.f.).

TuneraTaller: Esta plataforma web es pensada para que los talleres puedan organizarse mejor, tiene funciones como la gestión de clientes para mejorar la comunicación haciendo que la atención sea personalizada, el control de inventario y proveedores que sirve para garantizar un flujo de trabajo eficiente, y la administración del taller desde cualquier dispositivo (TuneraSoft, s.f.).

Yelp: Un sitio web que permite a los usuarios encontrar, calificar y reseñar negocios locales. Originalmente operaba solo como un sitio de restaurantes, pero ha crecido con el tiempo en los últimos años para incluir muchas más categorías, incluyendo salud, belleza, servicios para el hogar y sí, talleres de reparación de autos y motocicletas, entre otros. La plataforma también es accesible tanto a través de Internet como de aplicaciones para dispositivos móviles (Yelp Inc., s.f.).

2. PLANTEAMIENTO DEL PROBLEMA

Los motociclistas al encontrar dificultades para obtener un servicio técnico confiable y por consecuencia deben dedicar tiempo extra buscando un establecimiento. Como se mencionó en los antecedentes, entonces lo que pasa con ese tiempo es que se va en recorrer establecimiento tras establecimiento preguntando por la solución para su moto en particular, porque en varios casos se llega al establecimiento y no hay espacio, surgen imprevistos por parte del prestador de servicios y, así se va yendo el tiempo.

Algo que pasa mucho es que se llega al taller ya quizá programado para el trabajo que le van hacer, cierto trabajo no se puede hacer completo entonces toca volver de nuevo a buscar quién sí pueda hacerlo. Y no solo me ha pasado a mí, a cualquiera que necesite con urgencia le toca lo mismo. Entre tantas situaciones, uno acaba gastando más del dinero de la que tenía pensado. Y además de la plata es el desgaste que se genera después de todo el recorrido. Eso sin contar que después de dejar la moto, muchas veces a uno le toca llamar y llamar porque del taller no le dicen nada, ni si ya la empezaron a mirar ni si ya terminaron. Si se quiere que otro taller le dé una segunda opinión o le pase otra cotización se le suma más tiempo al proceso sin necesidad. también súmele que deba de ponerse a buscar repuestos por cuenta propia volvemos a lo mismo, más tiempo, y al final el motociclista es el que se lleva la peor parte. Todo eso junto hace que el motociclista termine gastando tiempo y energía de más.

hay amigos, amantes a las motos y los viajes en ellas, que han pasado por momentos difíciles en lugares donde no encuentra ningún servicio que pueda realizarse y cumplir con las expectativas, pasa con frecuencia y cuando no eres residente del área y sales de la ciudad. La incertidumbre sobre la calidad del servicio y la capacidad del establecimiento para satisfacer los requisitos de los usuarios y el servicio proporcionado por terceros, se convierten en una preocupación.

Así que al examinar lo anterior, pasamos horas buscando en varios talleres, incluso obteniendo información desactualizada a través de los navegadores, significa que debemos ir de un lugar a otro sin las piezas adecuadas y personas calificadas en el momento respectivo. Además, hay una falta permanente de confianza en la calidad del servicio proporcionado, particularmente cuando el mecánico de confianza no se encuentra por ningún lado, y el peligro de haber estado en una posición donde terminas pagando más por un trabajo mal hecho porque nadie se hace responsable cuando se culpan entre servicios de terceros. El proyecto es crítico para este fin, porque se trata de mejorar ese desorden para que el motociclista encuentre un servicio técnico bien establecido, rápido y totalmente transparente, para no desperdiciar dinero y tiempo. Como pregunta de investigación: ¿Como mejorar la gestión en la prestación de servicios informativos y de asistencia técnica para la reparación y el mantenimiento de motocicletas en el municipio de Rionegro, mediante la utilización de una plataforma de software móvil Android?

3. JUSTIFICACIÓN

Aquí está claro que hay oportunidad de mejora en cuanto a la información veraz, geolocalizada y actualizada sobre servicios de motocicletas. Consolidar la información y las características en la aplicación eliminará la incertidumbre, el tiempo y los esfuerzos innecesarios para buscar los servicios, y dará transparencia y seguimiento a los servicios técnicos para motocicletas.

La tecnología ha impactado directamente en el ser humano, creando un entorno de costumbres que permea el tejido de la sociedad. Por lo tanto, uno de los objetivos de esta aplicación móvil es ayudar a resolver este problema de tiempo interminable gastado en procesos ineficientes, para promover una cultura con mejores hábitos sobre el uso del tiempo, haciéndolo valer y luego gastarlo en lo que pueda ser más beneficioso para cada quien.

Los principales beneficios anticipados serían la optimización de la experiencia del usuario, donde la búsqueda de servicios técnicos para motociclistas sea fácil, y a la vez un sistema para que los establecimientos anuncien su especialidad y se comuniquen con los clientes, ofreciendo la mejor experiencia posible.

En temas financieros, inicialmente se propenderá por que el costo de las operaciones en producción sea sufragado por los establecimientos registrados, a cambio de los beneficios que esperan poder recibir con el uso de la aplicación.

Para mantener mensualmente los costos que demanda tener en funcionamiento de la infraestructura, se requieren aproximadamente 50 dólares, por lo que se debe lograr recaudar ese valor como mínimo.

Reducción de la ansiedad e incertidumbre: Con funciones como una evaluación diagnóstica preliminar, junto con información sobre la estancia del vehículo en el taller, los usuarios pueden relajarse o invertir el tiempo en otras actividades, ya que no hay temor inicial sobre cómo será el servicio para la motocicleta.

Tiempo ahorrado y huella de carbono: Al crear mapas y publicar detalles sobre el establecimiento, se evitan desvíos innecesarios para encontrar un taller ideal y así reducir la huella de carbono por el consumo innecesario del vehículo.

Transparencia: Las funciones de reseñas también apoyan la confianza, ayudando a mantener calidad en los servicios ofrecidos por el establecimiento.

4. OBJETIVOS

4.1. General

Implementar una plataforma de software móvil Android para la gestión de servicios informativos y de asistencia técnica para la reparación y el mantenimiento de motocicletas en el municipio de Rionegro.

4.2. Específicos

Ingeniería de Requisitos y Planificación:

Definir los requerimientos y la ingeniería de requisitos para la plataforma, identificando de manera precisa las necesidades y expectativas de los usuarios finales.

Diseño de Arquitectura:

Diseñar las estructuras y/o arquitecturas necesarias para la base de datos y el software.

Desarrollo de la Plataforma Móvil:

Desarrollar la aplicación de software móvil para el sistema operativo Android, enfocada en la gestión de servicios informativos y de asistencia técnica para el mantenimiento y reparación de motocicletas en el municipio de Rionegro.

5. MARCO TEÓRICO

El marco teórico de este proyecto se deriva del proceso de análisis de la gestión de servicios e información para la toma de decisiones, servicio de calidad y experiencia del usuario que permite comprender el contexto del problema que enfrentan los conductores de moto al buscar servicios de reparación y mantenimiento.

5.1 Gestión de servicios de mantenimiento y reparación

La gestión de servicios se define como el conjunto de actividades realizadas mediante el uso de la planificación, organización y control para satisfacer las necesidades y requisitos del cliente.

Se puede entender como una mala gestión, relacionada con el mantenimiento y reparación de motocicletas, se define como ambigüedad respecto a los servicios prestados, la disponibilidad de repuestos y la habilidad técnica del personal para tratar con modelos individuales de vehículos motorizados (Parasuraman et al., 1988).

Donde los establecimientos hacen una mala gestión de la información, el usuario ha tenido que migrar entre ubicaciones de manera que consume un recurso no renovable como lo es el tiempo, lo que genera más costos y frustraciones. Cuando un cliente solicita un servicio y este no se proporciona, ya sea por falta de suministros o

de personal capacitado, también revela una discrepancia entre lo que el establecimiento realmente ofrecía en relación con lo que el cliente quería.

5.2 Información confiable y toma de decisiones del usuario

La información confiable es una característica central de la toma de decisiones del usuario. Para el motociclista, es un componente fundamental cuando se trata de la determinación de un establecimiento. La disponibilidad de informes claros, actualizados y confiables sobre los servicios permite al usuario verificar qué servicios sí y cuáles no, con datos actuales y verificables sobre las opciones disponibles de servicios, ayudan a verificar que la elección de ese servicio conducirá realmente a clientes interesados, y por lo tanto la confianza en el taller.

Esto último, lleva a una probabilidad alta de un servicio no deseado. En este sentido, la falta de información centralizada se convierte en un problema de estructura, reduciendo el funcionamiento eficiente de la búsqueda y la contratación de servicios técnicos para la motocicleta.

5.3 Calidad del servicio y percepción del usuario

Según Parasuraman et al. (1988), la calidad del servicio se refiere a cómo el servicio proporcionado está alineado y es parte del cumplimiento de las promesas que ha hecho el establecimiento y lo que los clientes esperan. Dicho esto, en los servicios de reparación y mantenimiento de motocicletas, la calidad se basa no solo en los resultados técnicos finales, sino también en la transparencia con el diagnóstico, la comunicación con el cliente y la responsabilidad por posibles errores técnicos.

En este contexto, estas situaciones dañan la confianza del usuario y generan un nivel de riesgo al seleccionar un establecimiento, particularmente cuando el motociclista no tiene un mecánico confiable o se está moviendo en un municipio diferente al habitual. En este punto, mecanismos como las calificaciones y comentarios de otros usuarios permiten reducir esta incertidumbre, al ofrecer una referencia basada en experiencias previas.

5.4 Experiencia del motociclista en la búsqueda de servicios

La experiencia del usuario abarca todas las interacciones que el motociclista tiene durante la búsqueda, selección y recepción del servicio (Norman, 2013). Cuando la experiencia es mala se hace evidente, va de un lado a otro sin necesidad; no le dan información completa y queda con la duda de si el trabajo quedó bien hecho. Mejorar esa experiencia significa reducir los obstáculos al motociclista para encontrar lo que busca para su moto.

5.5 Transparencia y confianza en la prestación de servicios

Si un taller es transparente con lo que ofrece, el cliente confía más. Esto ocurre cuando se sabe qué le van a hacer a la moto, cuánto le va a costar y quién responde si algo sale mal. Cuando hay varios involucrados y ninguno asume la responsabilidad, ahí es donde empiezan los problemas y el cliente termina insatisfecho.

En este sentido, la transparencia es lo que hace que la relación entre el motociclista y el taller realmente funcione y con esto se logra que el mantenimiento de la moto deje de ser una lotería y se convierta en un proceso confiable, basado en la integridad de quien presta el servicio (Mayer et al., 1995).

5.6 Tiempo del motociclista

Desde el punto de vista del motociclista, el tiempo es un recurso valioso, ya que se consume en gran medida por las ineficiencias del proceso, desde la búsqueda del servicio hasta su contratación. Desde la teoría de la calidad del servicio (Parasuraman et al., 1988), el tiempo que el usuario se demora esperando o buscando un servicio afecta directamente como percibe la calidad de ese servicio. Si se demoró mucho en la búsqueda, ya la experiencia inicia de manera negativa.

5.7 Geolocalización y acceso móvil en la búsqueda de servicios

En el mercado hay numerosas soluciones que permiten la georreferenciación de lugares y que hoy en día las aplicaciones de mapas y navegación son ampliamente utilizadas y no siempre se han usado específicamente para el descubrimiento puntual de servicios especializados en los vehículos de cada persona.

El tener una aplicación móvil para esto, es realmente importante porque el motociclista necesita resolver este tipo de situaciones incluso mientras está en movimiento, por lo que se alinea con las dinámicas actuales del consumo masivo de información (Shneiderman et al., 2016).

Las herramientas de geolocalización cambiaron la manera de las personas a la hora de buscar y acceder a distintos servicios. Para este caso, al motociclista poder ubicar talleres cercanos a él, mejora la experiencia de búsqueda, le ahorra tiempo en desplazamientos innecesarios y le permite tomar decisiones según su ubicación (Goodchild, 2007).

5.8 Reseñas y calificación del servicio

Desde lo comercial, este tipo de plataformas representan una oportunidad para los establecimientos, ya que les permiten mostrar sus servicios, especialidades, generar mayor visibilidad y evaluar oportunidades de mejora y crecimiento para el negocio, facilitando la conexión con nuevos clientes.

Los comentarios y calificaciones de otros usuarios hacen parte importante del proceso de búsqueda de un servicio. En la práctica, las personas suelen basarse en la experiencia de otros usuarios antes de tomar la decisión, y más cuando se trata de un servicio técnico en manos desconocidas. Este comportamiento ha sido estudiado en el contexto de sistemas de reputación en línea, donde las opiniones de los usuarios suelen influir directamente en la percepción de la imagen de los establecimientos, en este caso, de los servicios ofrecidos (Resnick et al., 2000).

6. FASE 1. DISEÑO METODOLÓGICO

Desde la perspectiva de varias fases de un proceso iterativo (Pressman & Maxim, 2020), los resultados generados en el proceso de desarrollo permitieron la identificación y el análisis del negocio, la evaluación del conocimiento que ya se tenía, lo que permitió hacer ajustes oportunos antes de la implementación de la tecnología, a través los aspectos estratégicos, tácticos y técnicos.

6.1 Diseño estratégico

Antes de hacer cualquier línea de código para el negocio... esta fue la premisa básica de la primera fase. Los hallazgos luego evolucionaron a través de varias versiones de cada iteración, en las cuales se identificaron aspectos del dominio que no eran evidentes. Se pensó que se había definido un flujo en un momento, pero cuando llegaron las herramientas de descubrimiento, se supó qué regla faltaba, cuáles eran las necesidades de esos actores y qué no habíamos pensado incluir. Esta fase fue diseñada precisamente para apoyar el ajustar a tiempo. A nivel de pizarra los errores suelen ser más baratos que corregirlos en el código. Por lo tanto, se siguieron estrategias planificadas en este orden, cada una siendo una entrada para la otra y el conocimiento se solidificaba progresivamente.

6.1.2 Visión

Lo primero que se hizo fue describir la visión del proyecto. Para MotoGo, la visión llevó a identificar quiénes son los actores principales: el propietario de la motocicleta, el establecimiento que proporciona servicios de diagnóstico, reparación o mantenimiento, y el administrador del sistema, así como lo que necesita cada uno dentro del flujo del negocio. Este ejercicio proporcionó claridad del alcance del proyecto. Para más información sobre la visión, véase el Anexo 1, listado en la Tabla 2.

6.1.3 Mapa de Impacto

Habiendo identificado el alcance en el que se quería trabajar, se comienza con el mapa de impacto. Esta estrategia, propuesta por Adzic (2012), permite mapear los objetivos de nuestro proyecto en los pasos tangibles que se toman para abordar las preguntas: ¿Por qué lo estamos haciendo?, ¿Quién se ve afectado?, ¿Cómo haremos el impacto deseado? y ¿Qué necesitamos construir para lograrlo?

El Mapa de Impacto es un instrumento de planificación estratégica desarrollado por Adzic (2012) que vincula cuatro niveles de decisiones de planificación. El primero es el objetivo del negocio, derivado directamente de la visión. El segundo son los actores a quienes se pretende influir, es decir, las personas cuyo comportamiento dentro del proceso queremos cambiar. En este caso se identificó al propietario de la motocicleta, al representante del establecimiento y al administrador del sistema. El tercero son los impactos, el valor de este ejercicio ayudó a hacer filtros. Algunas funciones de esta presentación parecían buenas en algunos aspectos, pero cuando se medían contra el mapa, en realidad no trabajaban para alcanzar la primera versión. Para más detalle del mapa de impacto, véase el Anexo 2 listado en la Tabla 2.

6.1.4 Modelo de dominio

El modelo anémico en el Diseño Orientado al Dominio (Evans, 2003) es con el que se empieza. Esencialmente, se encontraron las entidades cruciales y sus características así como los vínculos entre ellas. En esa primera versión las entidades principales como Motocicleta, Sucursal, Diagnóstico, Servicio, y las estructuras... que

las apoyan: Marca, Línea, Modelo y Rango de Desplazamiento para describir una motocicleta; Ciudad, Departamento y Ubicación para localizar una sucursal; Horario, Detalle de Horario y Día para definir la disponibilidad de un taller. El modelo todavía no tiene reglas de negocio ni comportamiento como tal; corresponde a un esqueleto que muestra qué existe en el dominio y cómo se relaciona.

¿Entonces para qué sirve? Para hablar el mismo idioma. Antes de hacer este ejercicio, estábamos hablando literalmente de manera diferente sobre lo mismo: "taller" y "sucursal" se usaban indistintamente; "servicio" podría referirse tanto al catálogo de lo que ofrece un taller como a la ejecución concreta de un trabajo en una motocicleta. El modelo anémico obligó a encontrar un vocabulario común, que Evans (2003) describe como Lenguaje Ubicuo (Ubiquitous Language). Ese vocabulario definió los términos del dominio en español, los cuales se tradujeron al inglés para reflejarlos en el código fuente siguiendo las convenciones del lenguaje de programación. Para ver el gráfico relacionado con el modelo de dominio anémico véase el Anexo 3, listado en la Tabla 2.

6.1.4.1 Modelo enriquecido

El modelo enriquecido, dando vida al esqueleto del modelo de dominio dejó de ser anémico. Lo que se hizo fue tomar cada objeto del modelo inicial y enriquecerlo con toda la información necesaria para que pudiera guiar la implementación. Se definieron los atributos detallados de cada objeto de dominio con su formato y tipo de

datos, si es obligatorio o no, si se autogenera y si permite valores en blanco. También se asignaron sus responsabilidades, es decir, las operaciones que el objeto puede realizar dentro del negocio.

En términos concretos, para MotoGo esto significó definir, por ejemplo, la máquina de estados de los diagnósticos (SOLICITADO, EN PROCESO, COMPLETADO), las validaciones que cada entidad debe cumplir antes de ser persistida y los límites entre los módulos del sistema. Es importante aclarar que el modelo enriquecido fue una guía fundamental para la implementación, a un que no fue un documento estático. Durante el desarrollo surgieron atributos y relaciones, que en esta etapa de diseño no eran tan visibles, producto del entendimiento más profundo que se va adquiriendo cuando se trabaja con el código. Nunca se eliminó nada de lo definido en el modelo, pero sí se fueron agregando elementos. Con todo y eso, el ejercicio fue una parte fundamental del proceso pues dio la base sobre la cual construir con confianza. Los detalles relacionados con el modelo de dominio se pueden consultar en Anexo 3, listado en la Tabla 2.

6.1.5 Event storming

El descubrimiento de lo que no está a simple vista se aborda. Después de haber definido el vocabulario, se dio paso al Event Storming. Este método, propuesto por Brandolini (2021), consiste en identificar todos los eventos que ocurren en el negocio y ordenarlos cronológicamente. Un evento se refiere a algo que ya ha sucedido, ejemplos: "Diagnóstico solicitado", "Motocicleta registrada", "Servicio completado".

La fortaleza del Event Storming es considerar el verdadero camino del negocio en lugar de tener que pensar en pantallas o tablas de bases de datos. Si se mapea el evento "*Diagnóstico solicitado*" también se pregunta: ¿quién lo está causando? ¿Qué información se necesita? ¿Qué sucede después? ¿Puede fallar? De esas preguntas surgieron comandos, agregados, políticas y modelos de lectura que no habíamos pensado de otra manera.

Por ejemplo, se evidenció que el estado de un diagnóstico no era simplemente "pendiente" o "listo". Había estados intermedios como "en proceso" que tenían reglas particulares: un suario no puede cancela el diagnóstico en proceso, solo el establecimiento puede. Esas reglas no se habían visto cuando se hizo el modelo anémico, solo aparecieron en el trabajo de hacer el Event Storming. Los detalles relacionados con el Event Storming se pueden consultar en Anexo 4, listado en la Tabla 2.

6.1.6 Dimensionamiento del producto

Ya se tenía un mapa completo de lo que necesitábamos construir, con el modelo de dominio enriquecido. había algo que definir: *¿cuán grande es esto?*

El dimensionamiento es el proceso de dimensionar módulos y sus funcionalidades. Esto se realizó para evaluar la complejidad de cada parte basada en factores como el número de entidades, reglas de negocio a implementar, integraciones externas y nivel de interacción del usuario. No todos los módulos tenían el mismo peso de evaluación. Pero registrar a una persona es mucho más fácil que todo el proceso de un diagnóstico, que es ejecutado por un montón de actores, transiciones entre estados, validaciones cruzadas. El dimensionamiento dio una idea de cuán realista sería el esfuerzo e informó la priorización de lo que vendría después. Los detalles relacionados con el tallaje de las historias de usuario se pueden consultar en el Anexo 5 listado en la Tabla 2.

6.1.7 Mapa de historias de usuario

Ya en esta etapa se sabía qué construir y cuán grande debía ser cada pieza. Ahora era el momento de organizarlo de manera que tuviera sentido práctico. Se hizo esto con la ayuda del User Story Mapping (Patton, 2014), utilizando, un método propuesto por Patton (2014). El mapa se organizó en dos ejes. En el eje horizontal, se organizaron las áreas funcionales del sistema: Gestión del Tipo de Sitio, Gestión del Tipo de Servicio, Gestión del Servicio Realizado, Gestión de Franquicias, entre otros.

En el eje vertical, en la columna de la izquierda, se colocaron las versiones, de modo que cada fila representa una versión del producto. Las historias de usuario se distribuyeron dentro de esta cuadrícula según el área funcional a la que pertenecen y la

versión en la que se planeó su entrega. Las historias están ubicadas en las filas superiores que corresponden a las primeras versiones y son de mayor prioridad, mientras las de las filas inferiores estaban programadas para entregas posteriores.

El aspecto más útil de este ejercicio fue poder visualizar el producto completo de un vistazo. Además, sirvió para ver huecos funcionales: zonas donde faltaban historias de usuario, o donde el trabajo no estaba completo todavía. Aparte sirvió para ver cómo estaba repartido en versiones; si veíamos que una versión tenía muchas historias apretadas en una sola área, eso era señal de que tocaba redistribuir. Los detalles relacionados con el mapeo de las historias de usuario se pueden consultar en el Anexo 5 listado en la Tabla 2.

6.1.8 Product backlog

Del mapa de historias salió el Product Backlog, que básicamente es la lista con todas las historias del producto ordenadas por prioridad. Se le asignó un código a cada historia: HU01, HU02 y así sucesivamente; posteriormente se agruparon por el módulo funcional al que pertenecen. Para priorizar, se evaluó con base en qué genera el mayor valor de negocio para el usuario: lo primero que se aborda es lo que tiene

más sentido. El backlog no fue algo que ocurrió de repente. Se iba revisando a medida que se avanzaba y obteniendo información sobre el dominio. Historias que parecían simples se dividieron en varias, y otras que parecían complejas resultaron ser más sencillas de lo esperado. Los detalles relacionados con el Product Backlog se pueden consultar en el Anexo 5, listado en la Tabla 2.

6.1.9 Escritura de historias de usuario

Ya con el backlog armado, seguía escribir cada historia de usuario con todo el detalle. Se tomó como referencia el formato de Behaviour-Driven Development (BDD) (Wynne & Hellesøy, 2017), utilizando la estructura Gherkin —"Dado que... cuando... entonces..."— siguiendo el enfoque de desarrollo guiado por comportamiento, junto con los criterios de aceptación. Adicionalmente la estructura, se definió algo bastante útil al final y consistió en el código de los mensajes del sistema. En tanto para casos de éxito como de error, cada uno viene con un código de mensaje asociado (por ejemplo, MOD_B_REG_EXI_00001 para un registro exitoso de una sucursal). Se refieren directamente a las respuestas HTTP y mensajes que residen en la base de datos. No era la intención de hacer esto desde el primer día, sino que se convirtió en una necesidad cuando se evidenció que el mensaje de error estaba por todas partes y no era uniforme. Una vez que se hizo, la trazabilidad desde el requisito funcional hasta la respuesta que el usuario podría ver en pantalla estaba completamente formada. Los detalles relacionados con cada historia de usuario se pueden consultar en el Anexo 5, listado en la Tabla 2.

6.1.10 Plan de liberación

Para concluir este paso, cada lanzamiento tiene su enfoque, agrupa un conjunto coherente de historias de usuario disponibles para ser entregadas, probadas y demostradas como un incremento funcional del producto. Se definieron en tres criterios. En primer lugar, cada lanzamiento necesita un valor demostrable: debería ser posible mostrarlo a alguien y que esa persona entienda lo que se construyó. En segundo lugar, las historias dentro de un lanzamiento deben tener sentido ya que la misma entrega NO incluye la combinación de registro de personas con gestión de diagnósticos. En tercer lugar, se trata de resolver dependencias técnicas, es decir: si una historia depende de otra historia, ambas deben estar en el mismo lanzamiento o la dependencia en un lanzamiento anterior. Entre la estrategia y la ejecución estaba el plan de liberación que proporcionó un cronograma de entrega contra el cual se podía medir el progreso, y lo más importante, representaba puntos de contacto regulares de retroalimentación — puntos de inflexión donde se podía haber cambiado prioridades si algo cambiaba.

En retrospectiva, aquí había estado la fase que tomó más tiempo de análisis, pero también ahorró dificultades a largo plazo. Cada proceso alimentaba al siguiente, pues la visión definía el mapa de impacto, este a su vez dirigía el descubrimiento del dominio, este permitía dimensionar el producto. El dimensionamiento organizaba el mapa de historias, y las historias se escribieron y agruparon en un plan de entrega

concreto. Los detalles relacionados con el plan de liberación se pueden consultar en el Anexo 5, listado en la Tabla 2.

7. FASE 2. ARTEFACTOS TÉCNICOS

7.1 Diseño arquitectónico

Aquí se trabajó en el diseño técnico. Tomando todas las Historias de Usuario (HUs), determinamos los impulsores arquitectónicos, se validaron las tecnologías candidatas; comparamos alternativas de solución y se tomó la que mejor se adaptó a la solución. A partir de esa decisión, creamos el arquetipo de solución que convierte la estrategia en los componentes reales, y finalmente proponemos el conjunto de tecnologías para que todo lo anterior se refleje en las herramientas tecnológicas específicas.

7.1.1 Características de calidad

Las características de calidad se analizaron bajo el modelo propuesto por Bass et al. (2021), que permite visualizar no solo los requisitos técnicos sino también los que puede probar el usuario final. En este estudio, se llevó a cabo un taller de atributos

de calidad en el que se tiene en cuenta cada uno de los roles del sistema identificados en el mapa de impacto: Administrador, Representante del establecimiento y Usuario motociclista. A cada role se le asigna un voto, sobre un total de 120, entre 15 atributos de calidad.

En total se analizaron 135 variables correspondientes a 15 atributos. Los resultados mostraron que disponibilidad, confiabilidad y seguridad son tres atributos que se priorizaron más, seguidos por rendimiento y experiencia del usuario. Estos cinco atributos priorizados, fueron el filtro inicial para evaluar cada nueva decisión tecnológica tomada. Los detalles relacionados con el plan de liberación se pueden consultar en el Anexo 9, listado en la Tabla 2.

7.1.1.2 Escenarios de calidad

Se crearon escenarios de calidad basados en atributos y se utilizó la estructura SEI propuesta por Bass, Clements y Kazman (2021). Esto incluyó la definición de 71 escenarios: 36 básicos y 35 alternativos a través de 15 atributos de calidad orientados a que puedan ser probados por el usuario final. Cada escenario describe una situación específica bajo la cual un sistema necesita responder de una manera u otra, que tiene

los siguientes elementos: fuente del estímulo, estímulo, entorno, artefacto, respuesta y medida de respuesta. Los escenarios básicos proporcionan el comportamiento esperado bajo condiciones normales de trabajo, mientras que los escenarios alternativos dan el mecanismo de cómo se maneja si algo falla. Son estos escenarios los que más tarde se les da verificación y permiten probar que las tácticas y tecnologías seleccionadas realmente cumplen con lo que valoramos respecto a los atributos de calidad que tomamos en consideración. Los detalles relacionados con el escenarios de calidad se pueden consultar en el Anexo 9, listado en la Tabla 3.

7.1.1.3 Funcionalidades Esenciales

Las funcionalidades críticas se derivan de las historias de usuario, se agrupan en funcionalidades compuestas y se evalúan con base a dimensiones de impacto: económica, social, imagen de la entidad a la que representa y/o humana. En estas, se busca asegurar el funcionamiento mínimo reduciendo vulnerabilidades que serían perjudiciales para el núcleo de la aplicación, lo que podría crear efectos donde múltiples impactos podrían afectar a individuos, por ejemplo, caídas frecuentes de una aplicación en un contexto financiero puede provocar un daño de mala percepción en la imagen de la empresa por parte de sus clientes. Los detalles relacionados con el funcionalidades críticas se pueden consultar en el Anexo 10, listado en la Tabla 3.

7.1.1.4 Restricciones técnicas

En este caso, no hubo un cliente que pusiera restricciones tecnológicas para llevar a cabo el proyecto. Entonces, en ese apartado lo que se trabajó fueron temas de buenas prácticas de arquitectura, código limpio, seguridad, y testing. Los detalles relacionados con el restricciones técnicas se pueden consultar en el Anexo 11, listado en la Tabla 3.

7.1.1.5 Restricciones de negocio

En las restricciones de negocio se evaluaron los factores de riesgo que pudieran comprometer el curso del proyecto, retrasándolo o dejándolo con funcionalidades incompletas; que se tuvieran problemas de uso operativo o limitaciones económicas e igualmente la parte humana que suele ser bastante cambiante, como el número de personas que trabajan en el proyecto. Los detalles relacionados con las funcionalidades críticas, se pueden consultar en el Anexo 12, listado en la Tabla 2.

7.1.2 Spikes

Los spikes se derivan de los drivers arquitectónicos (Bass et al., 2021).

Es decir: se identifica un driver ej.: "necesitamos seguridad con OAuth2", entonces surge la duda técnica "¿Keycloak funciona con Flutter y Go?" y luego se hace un spike para validarlo con una prueba de concepto. Se evaluaron diferentes herramientas y tecnologías, revisando varios aspectos como la parte económica que fuera, preferiblemente de código abierto, la accesibilidad y la integración entre las que ya se iban seleccionando. Los detalles relacionados con las pruebas de concepto, se pueden consultar en el Anexo 13, listado en la Tabla 3.

7.1.3 Alternativas de solución

En esta fase, la alternativa de solución se escoge mediante el estudio previo de tácticas y estrategias, hecho en los drivers arquitectónicos y los spikes, estos ayuda a hacer la evaluación de las tres alternativas que se plantearon. Se termina escogiendo la alternativa 3, puesto que era la que cumplía con los atributos de mayor prioridad para el usuario final y que, a su vez, coincidía con los spikes que se adoptaron. Los detalles relacionados con las alternativas de solución, se pueden consultar en el Anexo 14, listado en la Tabla 3.

7.1.4 Arquetipo de solución

Dado lo anterior, cuando se habla de los elementos estratégicos y técnicos, ya podemos derivar el arquetipo de solución tanto del frontend como del backend. Esto

ya empieza a ser nuestra guía, que funciona como una traducción técnica de la necesidad del problema. Los detalles relacionados con el arquetipo de solución, se pueden consultar en el Anexo 15, listado en la Tabla 3.

7.1.5 Plataforma tecnológica

Si en el arquetipo se definió el plano (Hexagonal, BLoC, Capas, OAuth2/PKCE), que describe la estrategia, flujo de datos y separación de responsabilidades, aquí se presenta cómo se materializa (Go, Flutter, Keycloak, MySQL) la implementación y compatibilidad técnica. Los detalles relacionados con la plataforma tecnológica, se pueden consultar en el Anexo 16, listado en la Tabla 3.

7.2 Diseño detallado

En esta etapa corresponde a cómo se ve la solución del problema funcionando por dentro. Los diagramas muestran la estructura interna del sistema; a través de ellos se puede ver cómo se conectan las estructuras, cómo fluyen los datos, el modelo de datos relacional y cómo se hace el despliegue.

7.2.1 Diagrama de calses

El diagrama de clases, que en Go se traduce como un diagrama de estructuras, que sirve para tener la documentadas de las entidades del dominio, sus atributos y las relaciones entre ellas. Go no utiliza herencia sino composición mediante structs embebidos e interfaces, lo que se refleja en el diagrama como relaciones de composición en lugar de jerarquías de herencia.

Las entidades principales del dominio son las que representan a las personas con sus diferentes roles, las motocicletas con referencias técnicas, las sedes y franquicias, los horarios con sus excepciones, el catálogo de servicios, los diagnósticos y servicios realizados. Y cuando una entidad necesita datos de otra se usa composición. Los detalles relacionados con el diagrama de clases se pueden consultar en el Anexo 17, listado en la Tabla 3.

7.2.2 Diagrama de componentes

En este apartado se representa las unidades lógicas del proyecto, cómo interactúan, las relaciones y dependencias entre los diferentes módulos, para formar un sistema funcional.

Los componentes fueron identificados a partir de las funcionalidades del sistema, separando cada módulo de negocio como una unidad independiente. En el frontend, por ejemplo, están los módulos de autenticación, gestión de sedes, gestión de motos y diagnósticos, entre otros, y todos dependen de un componente Core compartido que maneja las comunicaciones y el estado. En el backend pasa algo similar: cada recurso del dominio como personas, sedes, motocicletas o servicios prestados es un componente con sus propias responsabilidades. Lo interesante del diagrama es que también muestra las dependencias entre módulos; por ejemplo, los diagnósticos dependen de motocicletas, y las calificaciones dependen de los servicios prestados. El diagrama unificado muestra ambas vistas y muestra cómo se comunican entre sí a través de la API REST y cómo se conectan con los servicios externos. El diagrama de componentes se puede consultar en el Anexo 18, listado en la Tabla 3.

7.2.3 Diagrama de paquete

Se realiza principalmente principalmente para tener una visión del código del proyecto más ordenada. Nos muestra cómo agrupar lógica y poder entenderlo de manera más organizada.

Los paquetes se organizaron siguiendo la arquitectura limpia de 4 capas (Martin, 2017), donde cada paquete agrupa un tipo de responsabilidad. Por ejemplo, un paquete se encarga de recibir las peticiones HTTP, otro de orquestar los casos de uso, otro de la lógica de negocio pura y otro de la persistencia en base de datos. Las entidades del dominio tienen su propio paquete aparte, organizado por recurso. En el frontend pasa algo parecido; cada funcionalidad tiene su carpeta con la parte visual, la lógica de negocio y los datos. Aparte hay un paquete compartido donde están las cosas que usan todas las features. Gracias a eso, cada pedazo de código se ocupa de lo suyo; y si llega alguien nuevo al equipo no le toca adivinar dónde queda cada cosa. El diagrama de componentes se puede consultar en el Anexo 19, listado en la Tabla 3.

7.2.4 Diagrama de secuencia

Este diagrama permite observar qué ocurre internamente cuando un usuario utiliza la aplicación. Básicamente, desde el momento en que el usuario le da tap a un botón se muestra la respuesta en pantalla. Se observa como y por dónde viajan los datos y qué validaciones hace el flujo antes de dar una respuesta.

Se tomó el registro de motocicleta como ejemplo de la secuencia. Hace parte de la HU43, ruta POST /motorcycles. ¿Por qué este caso de uso? Pues este pasa por

todas las capas y incluido el almacenamiento de las imágenes, que es un servicio externo y por lo tanto queda representativo. Comienza así: el usuario llena el formulario del registro de la motocicleta en la app y eso dispara una petición HTTP. viajan datos en JSON junto con el token JWT. Pero antes el debe pasar por el middleware: valida el token y revisa que el JSON esté bien armado. Si pasa, el handler toma el request y arma un DTO. El interactor lo recibe, lo convierte en una entidad de dominio y le pone el ID del propietario. Después se va al service. Ahí se revisa que la placa no esté registrada previamente, que la referencia exista y el formato de placa esté dentro de lo permitido. ¿Todo pasó? Entonces se persiste el registro en MySQL. Si en el registro se subió una foto, va al servicio de almacenamiento en la nube. Finalmente, la respuesta se regresa por el mismo camino hasta el usuario. El diagrama de secuencia se puede consultar en el Anexo 20, listado en la Tabla 3.

7.2.5 Diagrama de despliegue

Representa cómo queda todo en producción. Se cuenta con un VPS y se le instaló el servicio de Dokploy para facilitar los despliegues.

El servidor es Ubuntu 22.04 y Dokploy se encarga de orquestar todo lo que se despliega. Dokploy está pendiente de los webhooks de GitHub. Entonces cada *push* a la rama main genera la construcción de la imagen Docker y despliega el nuevo contenedor sin intervención manual. Traefik hace de proxy inverso: él decide a cuál servicio va cada petición y se encarga automáticamente de los certificados de

seguridad con Let's Encrypt. Y antes de que llegue cualquier petición al servidor, pasa por Cloudflare, gestiona el acceso y brinda protección ante ataques. En el VPS están corriendo varios contenedores, como el backend en Go, MySQL, Keycloak para la autenticación, y herramientas de monitoreo para poder revisar métricas y logs. Los servicios que están por fuera como servicios externos son consumidos por el backend cuando se necesitan. El diagrama de despliegue se puede consultar en el Anexo 21, listado en la Tabla 3.

7.2.6 Modelo entidad relación

Este diagrama es la foto de cómo están organizadas las tablas en la base de datos. Ahí se ven las entidades con sus campos, qué tipo de dato tiene cada uno, cuáles son llaves primarias y foráneas, y cómo se relacionan las tablas entre sí.

En MySQL las tablas de personas, donde cada persona puede tener rol de motociclista, de representante de un taller o de administrador. Después están las tablas de motocicletas con todo lo del catálogo técnico, las sedes con su ubicación en el mapa, los horarios y sus excepciones, los servicios que ofrece cada sede, los diagnósticos y los servicios realizados que van cambiando de estado: solicitado, en proceso y finalizado. Para las relaciones entre tablas se usa llaves foráneas con UUID, e información como las marcas y referencias se separan en tablas catálogo aparte para no repetir datos. Las tablas y columnas las nombran en inglés porque es el estándar

técnico, pero los datos que se guardan están en español porque al final los usuarios en su mayoría son colombianos. El modelo entidad relación se puede consultar en el Anexo 22, listado en la Tabla 3.

8. FASE 3: CONSTRUCCIÓN

En esta fase final se evaluó lo que ya se construyó. Con el producto en funcionamiento, la idea era medir la calidad de ese producto desde diferentes ángulos. Para eso se elaboraron cuatro artefactos: el análisis estático de código con SonarQube, las pruebas de carga, los patrones de diseño que se aplicaron durante el desarrollo y el manual de instalación.

8.1 Análisis estático de código

Este artefacto muestra ué tan sano o qué tanta deuda técnica hay en el código. se usó SonarQube para analizar tanto el backend en Go como el frontend en Flutter.

El Quality Gate pasó en ambos proyectos. Es decir que las condiciones mínimas de calidad se cumplieron: cero bugs, cero vulnerabilidades de seguridad, cero en calificación de mantenibilidad.

Con la cobertura de pruebas en el backend se tienen más de 530 tests automatizados con un promedio de 73.8% de cobertura en los paquetes de lógica de negocio. En frontend superó los 795 tests y llegó al 80% de cobertura. Ambos pasan el Quality Gate. El análisis estático de código se puede consultar en el Anexo 23, listado en la Tabla 3.

8.1.2 Pruebas de carga

Para las pruebas de carga se usó k6, que es la herramienta de Grafana para ese tipo de pruebas. La idea era saber cómo se comporta el backend bajo presión y encontrar los cuellos de botella antes de que los encuentren los usuarios.

El escenario que se probó fue local se inició con pocos usuarios virtuales y poco a poco subiendo hasta llegar a 1000 usuarios concurrentes durante unos minutos. Los endpoints probados fueron los más utilizados: autenticación, perfil, motocicletas, catálogos y búsqueda geográfica.

Los resultados fueron buenos. De casi un millón de peticiones, solo 3 fallaron. El percentil 95 del tiempo de respuesta quedó en 10.39ms y el percentil 99 en 45.25ms. Es decir que el 99.97% de las verificaciones fueron exitosas. El throughput

fue de 380 requests por segundo. El análisis de pruebas se puede consultar en el Anexo 24, listado en la Tabla 4.

8.1.3 Patrones de diseño

Acá se documenta los patrones que se iban usando durante el desarrollo. Estos fueron apareciendo por los problemas que salían.

En el backend se usó arquitectura de cuatro capas (Martin, 2017) para organizar el código en diferentes niveles de responsabilidad. El acceso a datos, desde el inicio se mantuvo de manera separada (Fowler, 2002), mientras tanto una capa llamada Interactor como intermedia entre el cliente y los servicios coordinaba la lógica principal del sistema, lo que facilitó la organización de los componentes.

En el frontend fue, clean architecture de tres capas, y manejo de estados de la ui, se hizo el manejo reactivo de la información. los patrones de diseño se puede consultar en el Anexo 31, listado en la Tabla 4.

8.1.4 Manual de instalación

El manual se elaboró para que se pueda montar el proyecto de manera guiada. Cubre backend y frontend más las bases de datos, ya sea vacías o con la información

necesaria para el funcionamiento de la aplicación. El manual de instalación se puede consultar en Tabla 5 *Anexos de puesta en marcha del proyecto MotoGo*.

8.2 Devops

El proyecto en producción. Se montó toda una infraestructura basada en contenedores y despliegue continuo, siguiendo los principios de entrega continua (Humble & Farley, 2010). La idea principal era que el sistema se ejecutara de forma estable, monitoreada, sin depender de procesos manuales cada vez que había un cambio.

Infraestructura de hosting:

Se configuró todo en un VPS con Ubuntu 22.04 como sistema operativo base. Se trata de un servidor con 6 vCPUs y 12 GB de RAM, suficiente para ejecutar los diferentes servicios del proyecto sin pisarse unos a otros. Para la administración del servidor se utilizó Dokploy, que es una plataforma PaaS autohospedada y permite gestionar implementaciones sin tener que entrar vía SSH para configurar algo. La información de devops se puede consultar en el Anexo 30, listado en la Tabla 4.

9. RESULTADOS

Los resultados que aquí se presentan van más enfocados a mostrar el funcionamiento del producto y ver cómo se resolvió la pregunta de investigación: *¿Como mejorar la gestión en la prestación de servicios informativos y de asistencia técnica para la reparación y el mantenimiento de motocicletas en el municipio de Rionegro, mediante la utilización de una plataforma de software móvil Android?*

La respuesta a esa pregunta: Es una plataforma móvil Android que conecta a los motociclistas con los establecimientos y tiendas de motos, facilitando el descubrimiento de servicios cercanos, la gestión de diagnósticos y el seguimiento completo del historial de servicios.

9.1 Perspectiva del motociclista

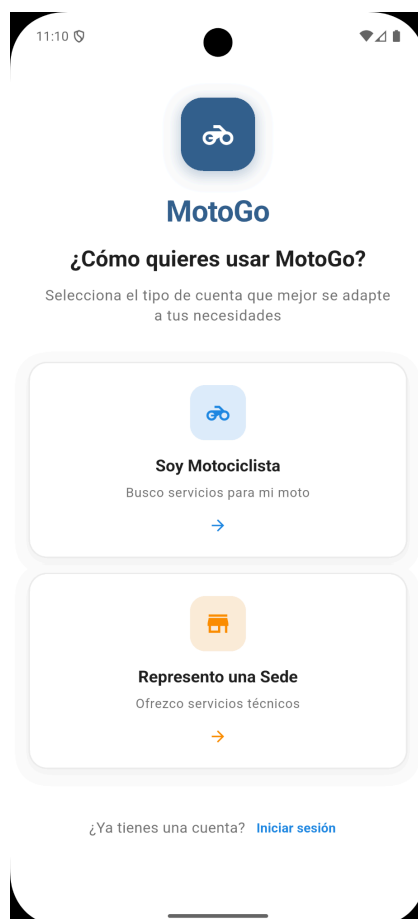
El motociclista es el usuario principal. Desde su perspectiva, la aplicación le permite registrar motos, conocer establecimientos cercanos, solicitar diagnósticos, ver la oferta de servicios y calificarlos de manera individual.

9.1.1 Bienvenida y registro

Ahora bien, al abrir la aplicación por primera vez, el usuario se encuentra con la pantalla de bienvenida allí puede elegir su rol: ingresar como motociclista o como representante de la sede (ver Figura 1).

Figura 1

Pantalla de bienvenida con selección de rol



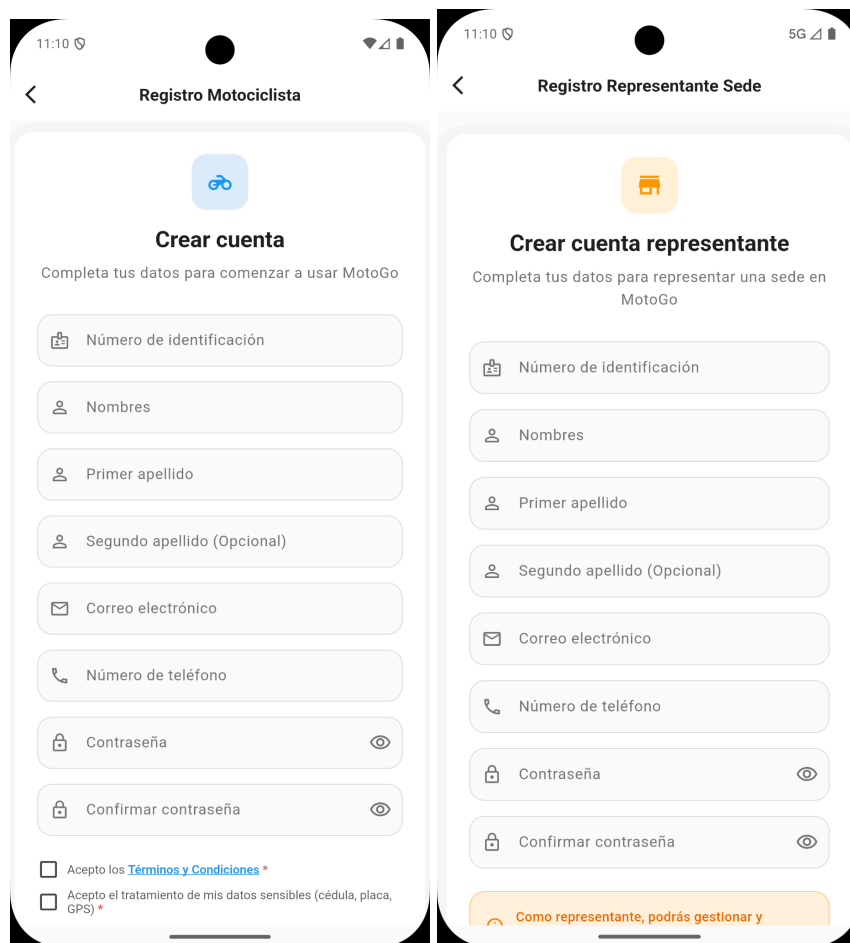
El formulario de registro de motociclistas recoge la información necesaria y validada (Figura 2). Del mismo modo, existe un formulario para los representantes de la sede. (Figura 3).

Figura 2

Formulario de registro para motociclista y

Figura 3

Formulario de registro de representante de sede



9.1.2 Autenticación

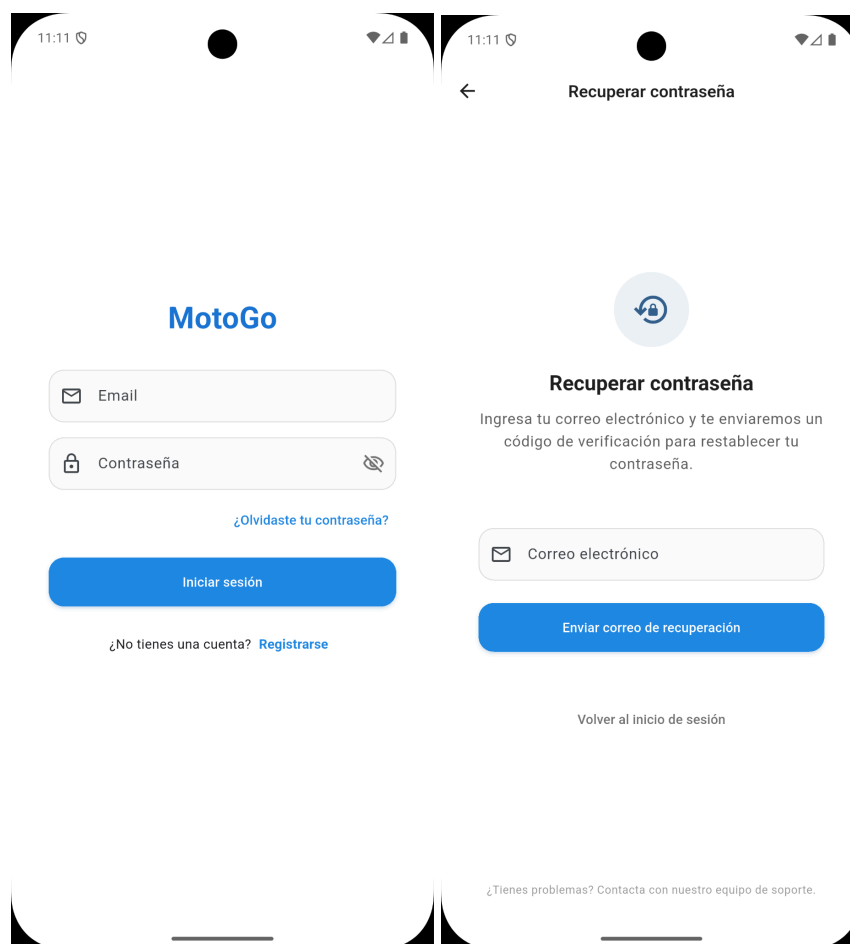
En la práctica, el sistema cuenta con un flujo de autenticación que incluye inicio de sesión (Figura 4) y recuperación de contraseña (Figura 5). El usuario puede restablecer su contraseña a través de un proceso basado en el correo electrónico.

Figura 4

Pantalla de inicio de sesión

Figura 5

Pantalla de recuperación de contraseña

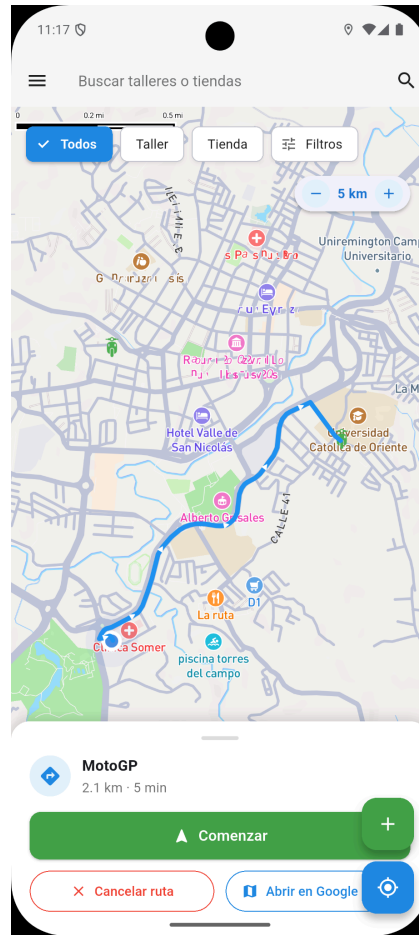


9.1.3 Descubrimiento de sedes en el mapa

Una vez autenticado, el motociclista accede a la pantalla home: donde puede ver un mapa interactivo que muestra ubicaciones cercanas (Figura 6). Este mapa utiliza geolocalización en tiempo real y ofreciendo filtros avanzados para que el usuario pueda encontrar el tipo de taller que necesita su moto.

Figura 6

Pantalla con mapa interactivo de descubrimiento de sedes con filtros por marca, cilindraje y tipo de establecimiento



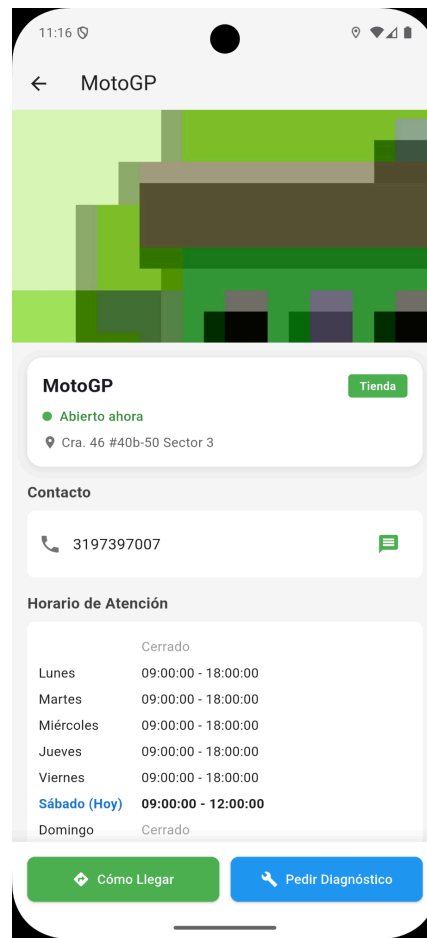
Esta funcionalidad responde directamente a la pregunta de investigación. El problema era que los motociclistas no tenían una forma eficiente de encontrar talleres que dieran servicio a su tipo de motocicleta. Con el mapa y los filtros por marca y cilindrada, el usuario sólo ve los establecimientos que realmente lo atienden, mejorando así la gestión en la prestación de servicios de información.

9.1.4 Detalle de sede

En la práctica, al seleccionar una ubicación en el mapa, el usuario accede a su perfil completo organizado en secciones: información general, servicios ofrecidos, horario de atención, ubicación exacta y opiniones de otros usuarios (Figura 7).

Figura 7

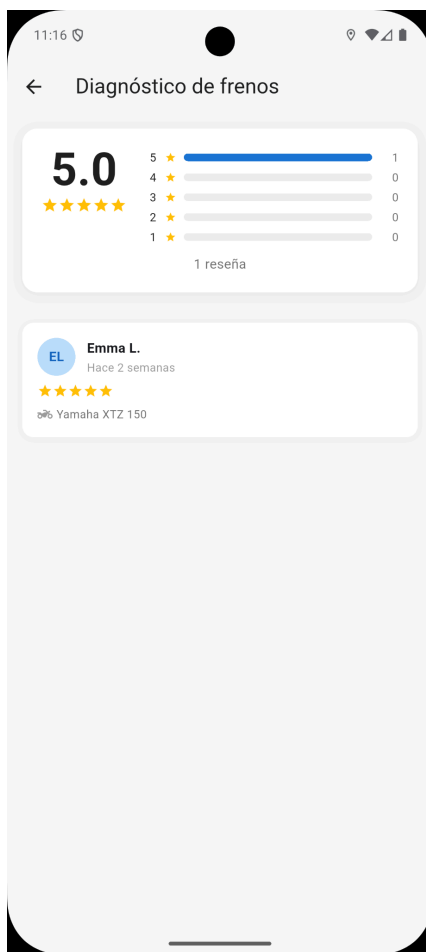
*Detalle de sede con información general, servicios ofrecidos y
horarios de atención*



El sistema de valoraciones permite a los motociclistas ver las calificaciones y reseñas que han dejados por otros usuarios, reduciendo incertidumbres antes de elegir un establecimiento (Figura 8).

Figura 8

Sección de reseñas y valoraciones de otros motociclistas



9.1.5 Gestión de motocicletas

El motociclista puede registrar múltiples vehículos en su perfil, cada uno con información detallada como la marca, línea, cilindrada, año y kilometraje (Figura 9). Permitiendo a la plataforma personalizar las recomendaciones y a los talleres conocer con antelación las características del vehículo.

Figura 9

Formulario de registro de una nueva motocicleta

11:12

← Registrar Motocicleta

Información Básica
Registra los datos de tu motocicleta

Imagen de tu Moto

Toca para cambiar la imagen
(Opcional)

Placa

Seleccionar referencia

Año

Kilómetros Actuales

Notas personales

0/500

Registrar Motocicleta

En la Figura 10 se muestra la lista de motocicletas que han sido registradas por el usuario, mientras que la Figura 11 se presenta el formulario de edición donde se pueden actualizar datos de la motocicleta.

Figura 10

Lista de motocicletas registradas en el perfil del usuario y

Figura 11

Formulario de edición de datos de una motocicleta

The image displays a mobile application interface for editing motorcycle data. It is split into two main sections: a list of motorcycles on the left and a detailed edit form on the right.

Left Panel: Mis Motocicletas

- ZZT97T**: Año: 2022 | 98k km. Note: "Pendiente cambio de kit de arrastre".
- ABC123**: Año: 2024 | 15.5k km. Note: "Actualizado desde Bruno - HU44 te...".
- IMG456**: Año: 2024 | 1k km. Note: "Moto con imagen de perfil".
- MRC35E**: Año: 2027 | 5k km. Note: "ok".

Right Panel: Editar Moto

Imagen de tu Moto: A photo of a motorcycle.

Placa: MRC35E. Below it, a message: "La placa no se puede modificar".

Año (opcional): 2027.

Kilometraje actual (opcional): 5000 km.

Notas (opcional): ok.

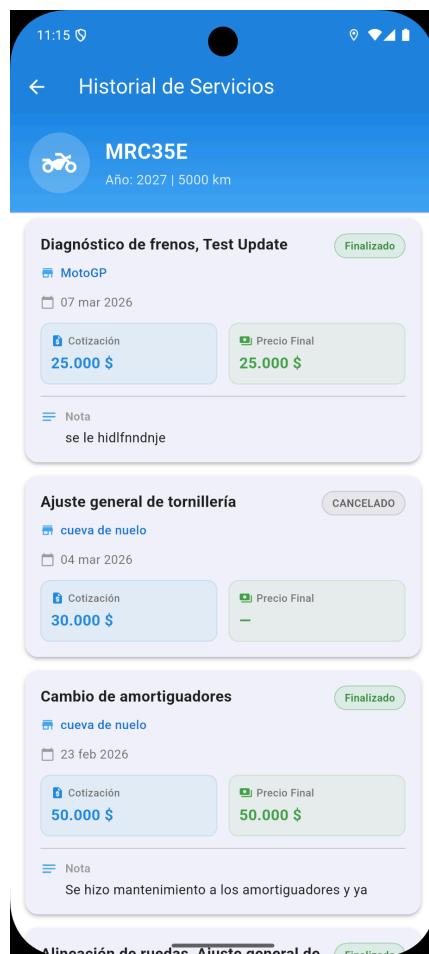
Guardar Cambios: A blue button at the bottom.

9.1.6 Historial de servicios y calificaciones

Cada motocicleta cuenta con un historial de servicios realizados, con el estado de cada uno ya sea Solicitado, En Proceso, Completado o Cancelado e información de costos (Figura 12).

Figura 12

Historial de servicios realizados a una motocicleta



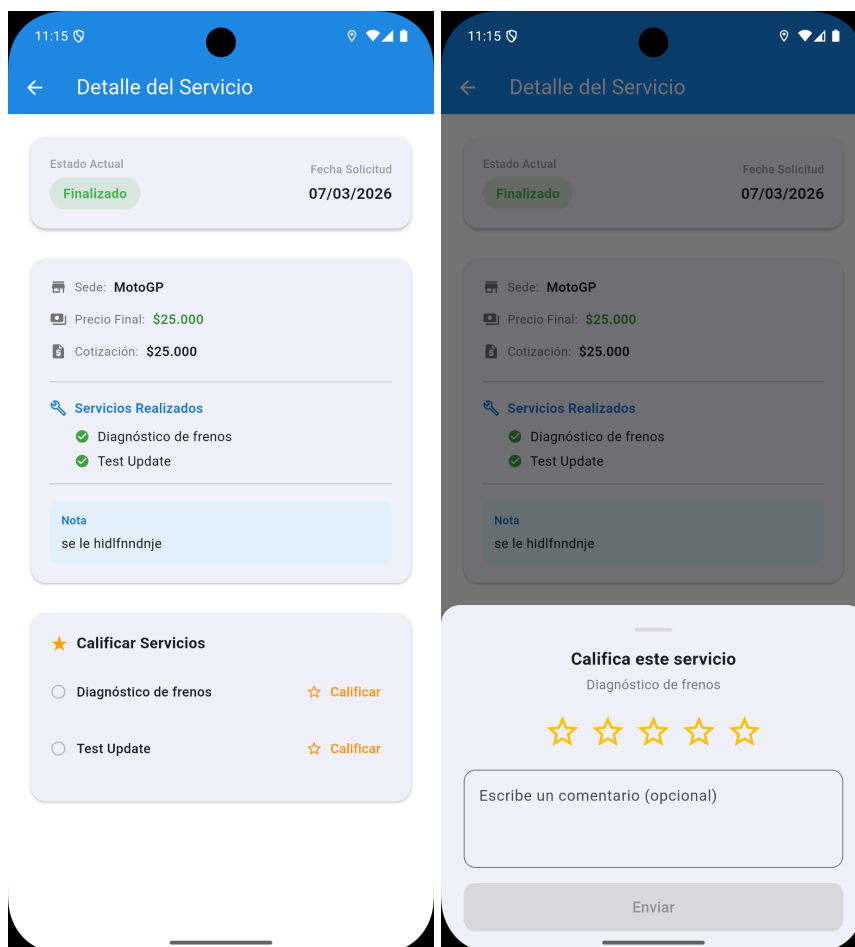
Al finalizar marcar un servicio como finalizado, el motociclista queda ahbilitado ver el detalle completo (Figura 13) y poner una calificación de estrellas y un comentario (Figura 14), alimentando el sistema de reseñas que ayuda a otros usuarios a tomar mejores decisiones.

Figura 13

Detalle de un servicio marcado como finalizado, con información de costos

Figura 14

Sistema de calificación con estrellas y campo de comentario



9.1.7 Perfil y configuración

El usuario gestiona su información personal desde la sección de perfil (Figura 15), donde podrá actualizar sus datos y cambiar su contraseña (Figura 16). El menú lateral da acceso a todas las secciones de la aplicación (Figura 17).

Figura 15

Pantalla de perfil del usuario con sus datos personales

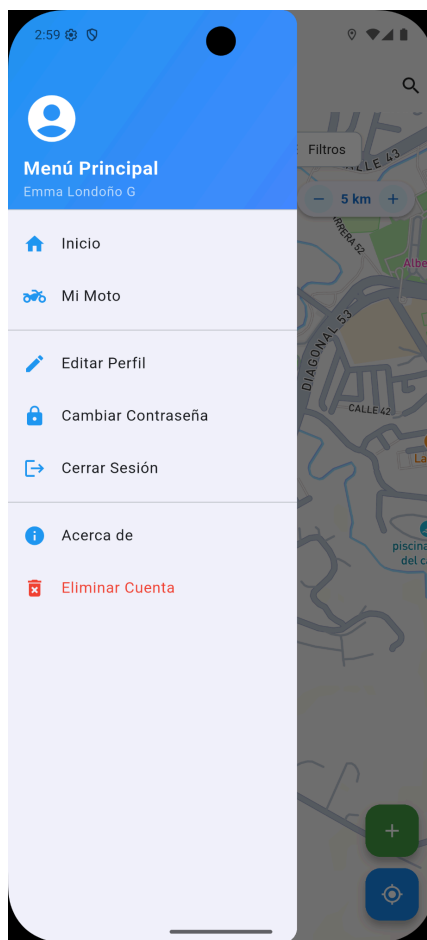
Figura 16

Pantalla de cambio de contraseña.

The image displays two side-by-side screenshots of a mobile application interface. The left screen, titled 'Editar mis datos', features a user profile icon at the top, followed by the title and subtitle 'Actualiza tu información personal'. Below this are several input fields: 'Número de identificación' (1036956627), 'Nombres' (Emma), 'Primer apellido' (Londoño), 'Segundo apellido (opcional)' (G), 'Correo electrónico' (negro@yopmail.com), and 'Número de teléfono' (3113633901). A 'Guardar cambios' button is at the bottom, with a message 'No hay cambios para guardar' below it. The right screen, titled 'Cambiar Contraseña', features a lock icon at the top, followed by the title and subtitle 'Ingresa tu contraseña actual y define una nueva contraseña para tu cuenta.' Below this are three input fields: 'Contraseña actual', 'Nueva contraseña' (with a note 'Mínimo 8 caracteres, una mayúscula, una minúscula y un número'), and 'Confirmar nueva contraseña'. A blue 'Cambiar Contraseña' button is at the bottom.

Figura 17

Menú lateral de navegación de la aplicación



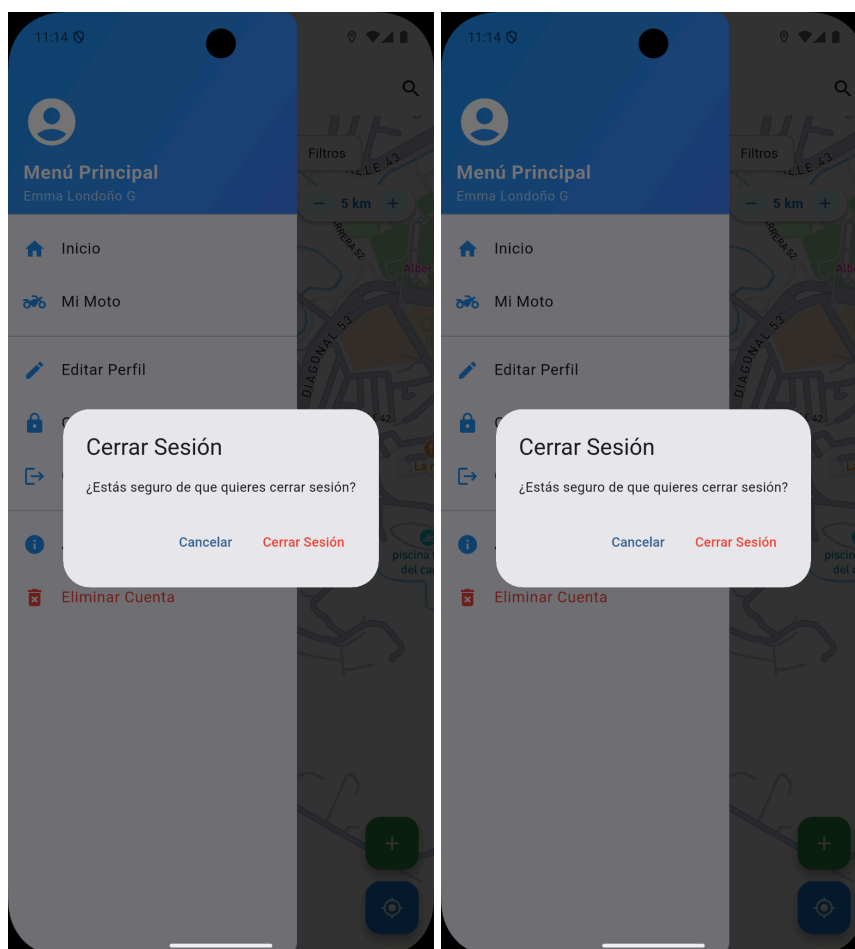
Por seguridad, tanto el cierre de sesión (Figura 18) como la eliminación de la cuenta (Figura 19) requieren una confirmación explícita por parte del usuario.

Figura 18

Diálogo de confirmación para cierre de sesión

Figura 19:

Diálogo de confirmación para eliminación de cuenta



9.1.8 Solicitud de diagnóstico

Desde la vista de detalle de una sede, el usuario puede navegar a solicitar un diagnóstico para su motocicleta. El formulario de solicitud (Figura 20) permite seleccionar una de las motocicleta registradas, describir el problema que presenta, adjuntar fotografías desde distintos ángulos (frontal, trasera y lateral), y otorgar permisos de consulta para que la sede pueda buscar la moto por placa. Al final del

formulario se muestra una vista previa del mensaje que será enviado por WhatsApp a la sede seleccionada.

Figura 20:

Formulario de solicitud de diagnóstico con selección de motocicleta, descripción del problema, evidencia fotográfica y vista previa del mensaje.

The screenshot shows a mobile application interface for a motorcycle diagnostic request. The title bar is blue with a back arrow and the text 'Diagnóstico para MotoGP'. The form consists of several sections:

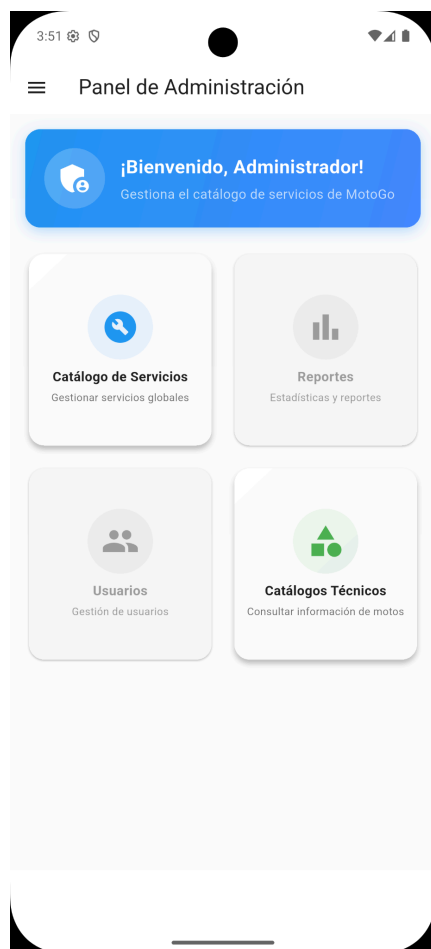
- Mi Moto:** A section with a blue header. It contains a dropdown menu labeled 'Selecciona tu moto' with the selected value 'MRC35E - 2027'.
- Describe el problema:** A section with a blue header. It contains a text input field with the placeholder text 'Escribe aquí los detalles del problema que presenta tu moto...'.
- Adjuntar fotos (opcional):** A section with a blue header. It contains three photo thumbnails labeled 'Frontal', 'Trasera', and 'Lateral', each with a red 'X' icon. There is also a plus sign icon for adding more photos.
- Permisos de consulta:** A section with a blue header. It contains a toggle switch labeled 'Permitir consultar tu moto por placa a MotoGP'. Below the toggle, there is a description: 'Al activar, la sede podrá buscar tu motocicleta usando la placa para revisar el historial de diagnósticos.' The toggle is currently turned on.
- Vista previa del mensaje:** A section with a blue header. It contains a WhatsApp message preview with a green circular icon with a white telephone handset. The message text is: 'Hola MotoGP, solicito un diagnóstico para mi moto: Placa: MRC35E'.

9.2 Perspectiva del administrador

El administrador tiene acceso a un panel para gestionar los catálogos técnicos y el catálogo de servicios que alimentan la plataforma. Al iniciar sesión, se puede acceder al Panel de Administración con acceso a los módulos de Catálogo de Servicios y Catálogos Técnicos (Figura 21).

Figura 21

Panel de Administración con accesos a Catálogo de Servicios y Catálogos Técnicos

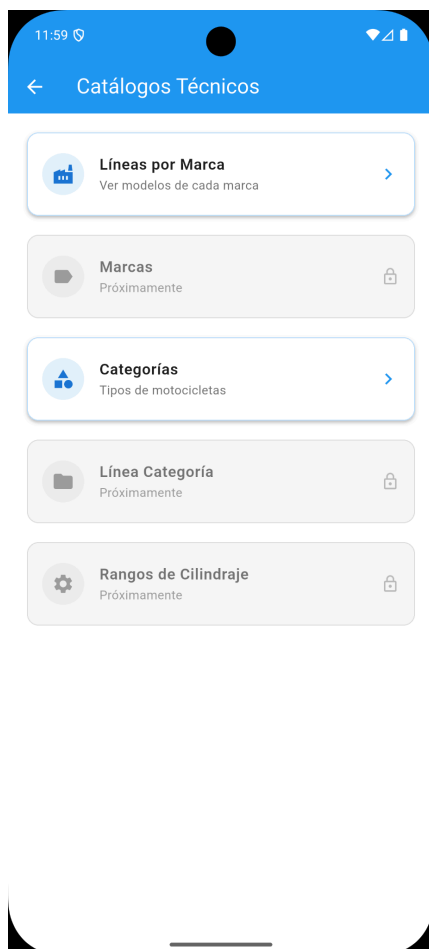


9.2.1 Catálogos técnicos

Al ingresar al módulo de catálogos técnicos, el administrador encuentra un menú con los diferentes catálogos disponibles: Líneas por Marca, Marcas, Categorías, Línea Categoría y Rangos de Cilindraje (Figura 22).

Figura 22

Menú principal de Catálogos Técnicos con los módulos disponibles.



9.2.2 Líneas por marca

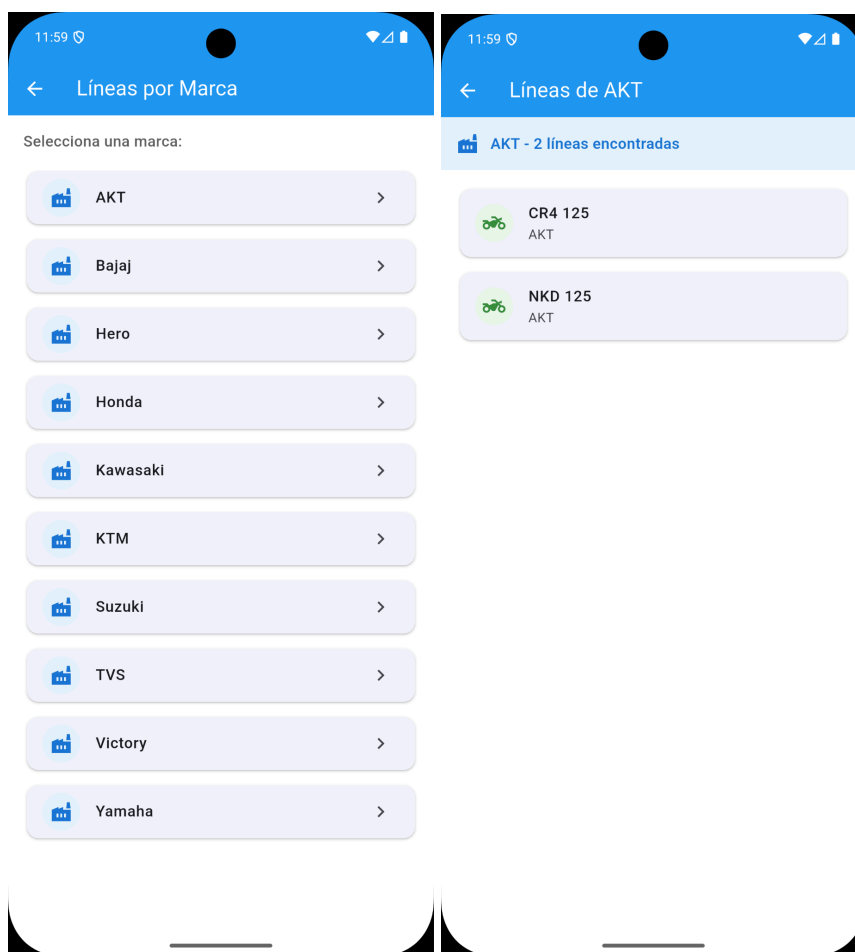
El catálogo de Líneas por Marca permite consultar las líneas de motocicleta organizadas por marca. En la Figura 23 se muestra la lista de marcas disponibles y, al seleccionar una marca se despliegan sus líneas con modelo y cilindraje (Figura 24).

Figura 23

Lista de marcas de motocicletas disponibles en el catálogo

Figura 24

Líneas de motocicleta asociadas a la marca.



9.2.3 Categorías de motocicletas

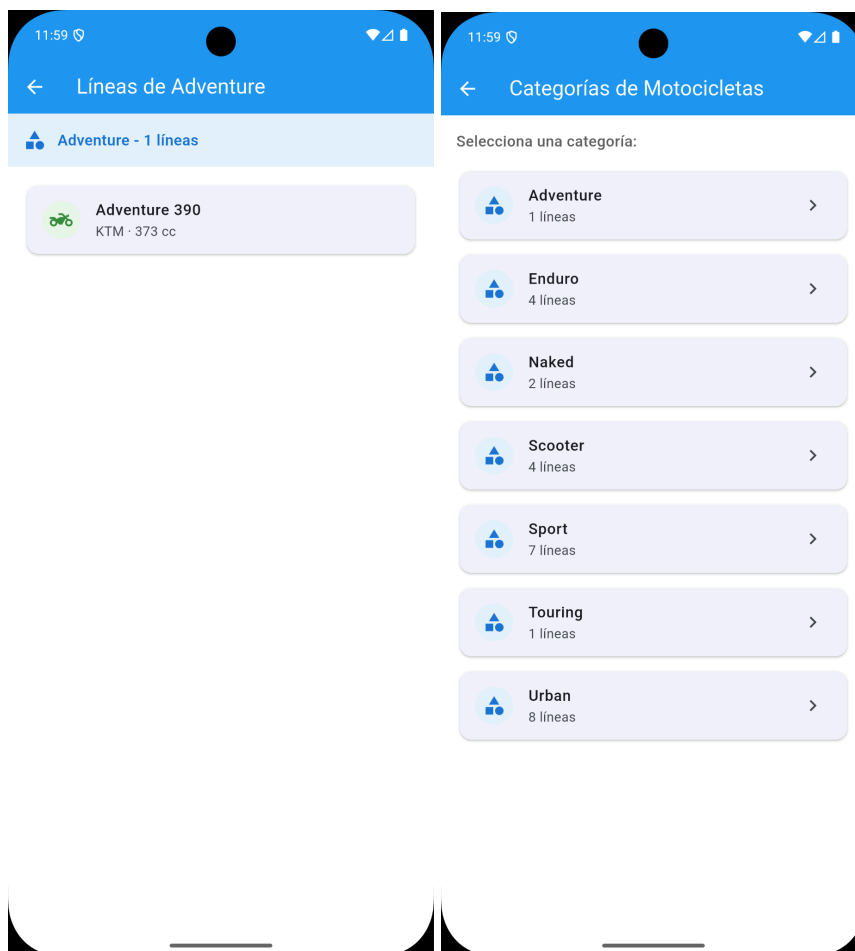
El catálogo de categorías clasifica las motocicletas por tipo: Adventure, Enduro, Naked, Scooter, Sport, Touring y Urban, cada una con la cantidad de líneas asociadas (Figura 25). Al seleccionar una categoría, se muestran las líneas que pertenecen a ella con su marca y cilindraje (Figura 26).

Figura 25

Lista de categorías de motocicletas con cantidad de líneas por categoría

Figura 26

Líneas de motocicleta dentro de la categoría Adventure.



9.2.4 Catálogo de servicios

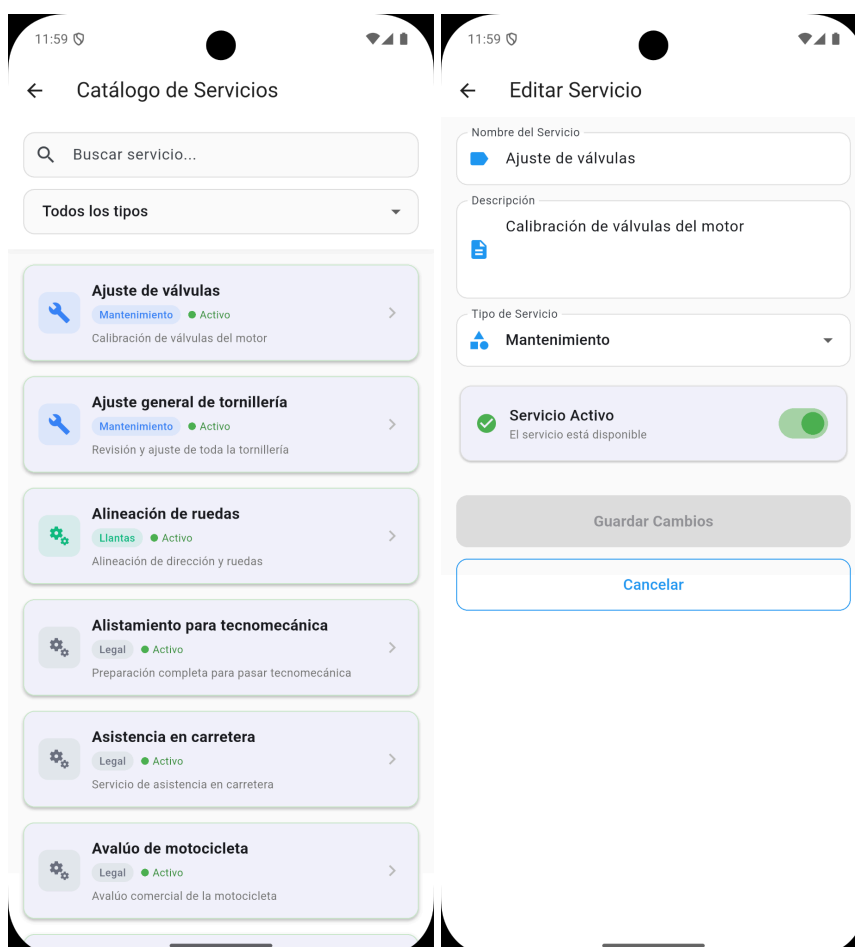
El administrador gestiona el catálogo de servicios disponibles en la plataforma (Figura 27). Cada servicio tiene nombre, descripción, tipo y un indicador de estado. Los servicios se pueden buscar y filtrar por tipo. Al seleccionar un servicio, se accede al formulario de edición donde se puede modificar el nombre, la descripción, el tipo y el estado de disponibilidad (Figura 28).

Figura 27

Catálogo de servicios con buscador, filtro por tipo y estado de cada servicio

Figura 28

Formulario de edición de un servicio con nombre, descripción, tipo y toggle de activación



9.3 Perspectiva del representante de sede

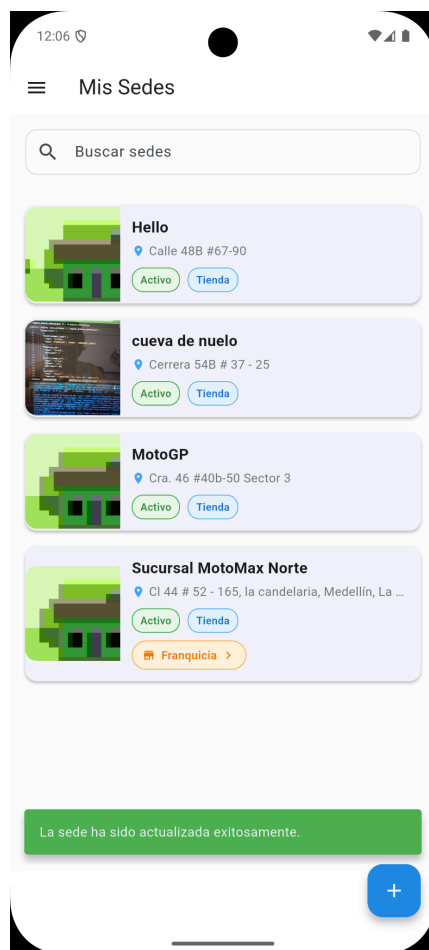
El representante de la sede puede gestionar todo lo relacionado con su establecimientos es decir: servicios, horarios, localización geográfica, diagnósticos y servicios realizados en motocicletas.

9.3.1 Mis sedes

Al iniciar sesión, el representante ve la lista de sus sedes con información resumida: nombre, dirección, estado y tipo (Figura 29). Las sedes que pertenecen a una franquicia están marcadas con una etiqueta especial.

Figura 29

Lista de sedes del representante con buscador y etiquetas de franquicia.



9.3.2 Perfil de la sede

Cada sede tiene un perfil completo organizado en pestañas: Servicios, Horarios, Ubicación y Editar. El diseño permite al representante gestionar todos los aspectos de su establecimiento de forma organizada.

pestaña de Horarios:

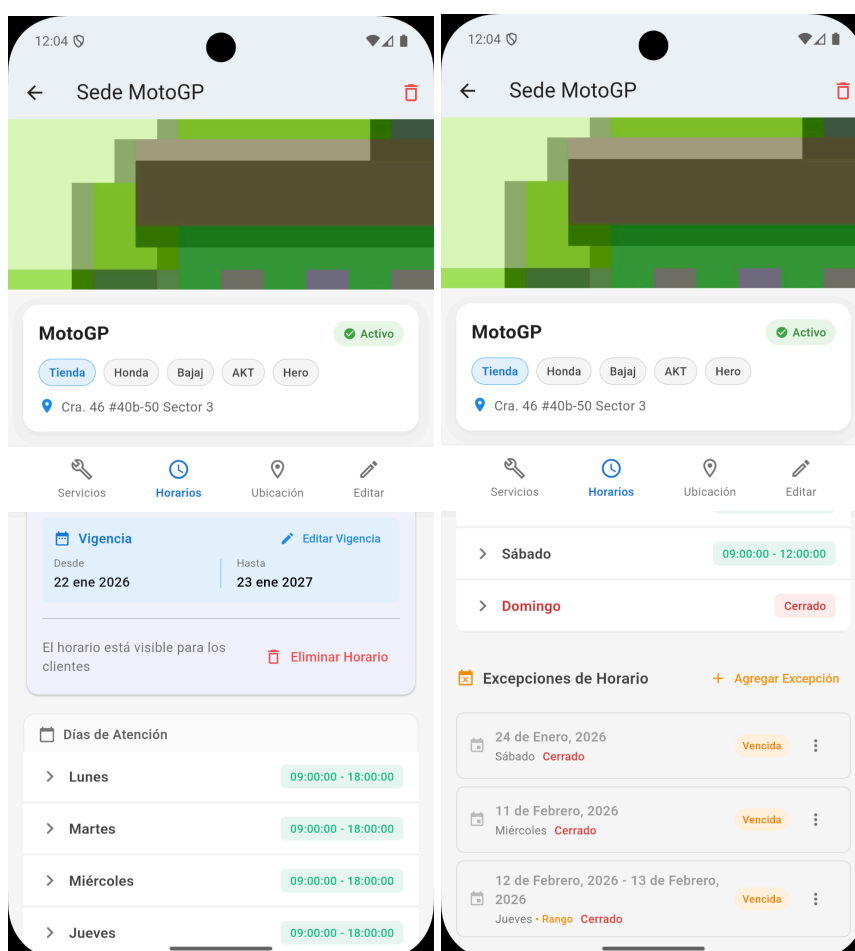
Muestra la vigencia del horario, los días de atención con su horario de apertura y cierre (Figura 30), y las excepciones de horario como feriados o cierres definidos por el representante de con su estado de vigencia (Figura 31).

Figura 30

Gestión de horarios de atención con vigencia y días de operación

Figura 31

Excepciones de horario con indicador de estado vencido.

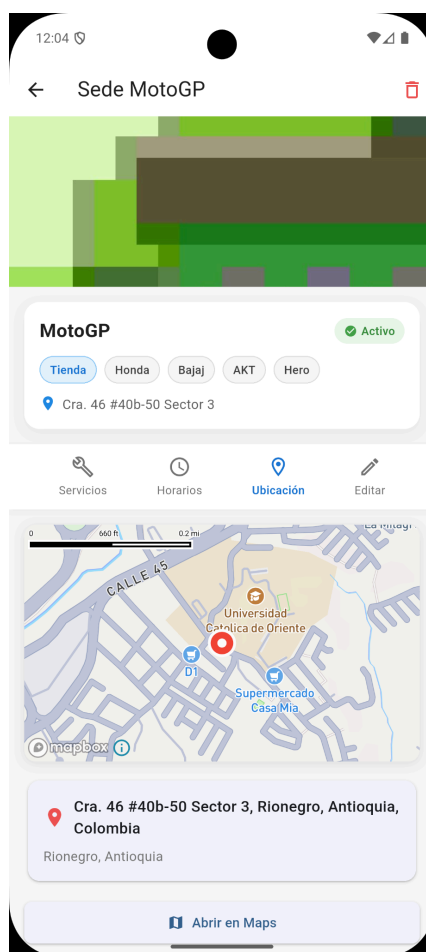


pestaña de Ubicación:

Se presenta un mapa con la ubicación de la sede, la dirección completa y un botón para abrir la navegación en la aplicación de mapas del dispositivo (Figura 32).

Figura 32

Ubicación de la sede.



pestañas de Edición:

Permite modificar toda la información de la sede: imagen, nombre, tipo de establecimiento, dirección, departamento, ciudad, marcas que maneja y rangos de cilindrada (Figura 33).

Figura 33

Formulario de edición de la información de una sede.

12:04

← Editar Sede

Imagen de la Sede

Nombre de la Sede

MotoGP

Tipo de Establecimiento

Tienda

Dirección

Cra. 46 #40b-50 Sector 3

Departamento

Antioquia

Ciudad

Rionegro

Marcas que maneja

✓ AKT ✓ Bajaj ✓ Hero ✓ Honda

Kawasaki KTM Suzuki TVS Victory

Yamaha

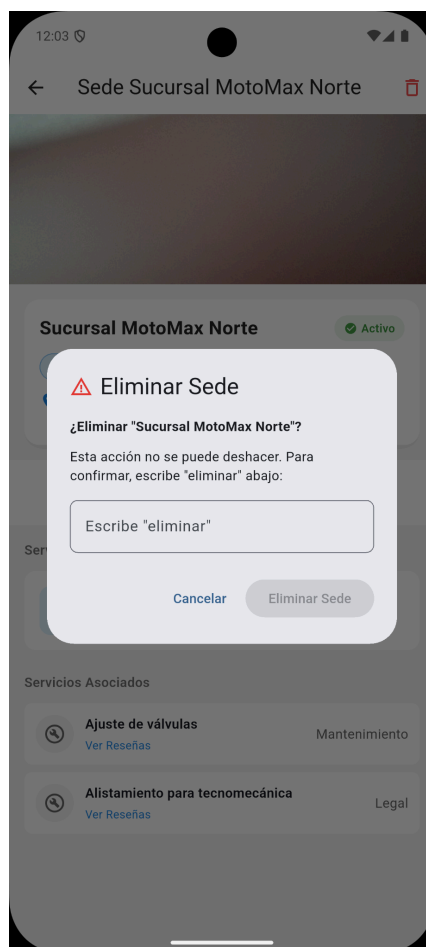
Rangos de Cilindraje que atiende

✓ Bajo (50-200cc) Medio (200-500cc)

La eliminación de una sede requiere confirmación de seguridad: el representante debe escribir "eliminar" manualmente para evitar eliminaciones accidentales (Figura 34).

Figura 34

Diálogo de confirmación de seguridad para eliminación de sede.



9.3.3 Creación de nueva sede

El representante puede crear una nueva sede con un formulario completo que incluye imagen, datos de ubicación con selección en cascada de departamento, ciudad, selección de marcas y, rangos de cilindraje que atiende (Figura 35).

Figura 35

Formulario de creación de una nueva sede.

12:05

← Crear Sede

Imagen de la Sede

Toca para agregar una imagen
(Opcional)

Nombre de la Sede

Tipo de Establecimiento

Dirección

Departamento

Selecciona un departamento

Primero selecciona un departamento

Marcas que maneja

AKT Bajaj Hero Honda Kawasaki

KTM Suzuki TVS Victory Yamaha

Selecciona al menos una marca

Rangos de Cilindraje que atiende

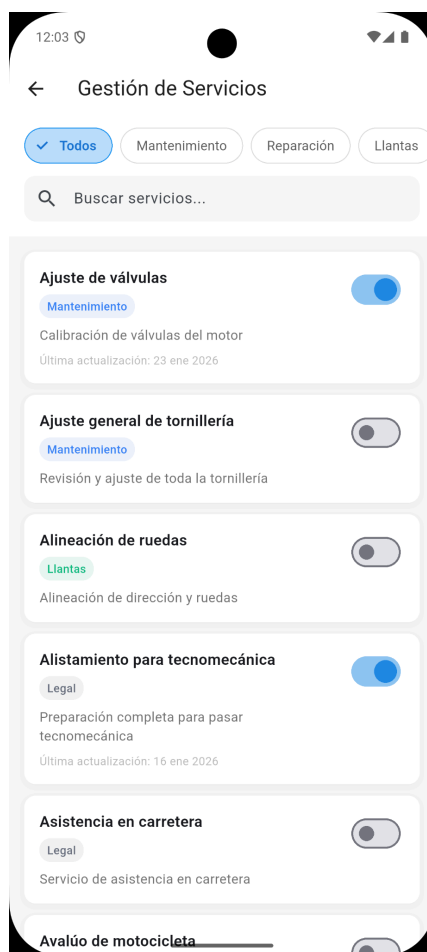
Bajo (50-200cc) Medio (200-500cc)

9.3.4 Gestión de servicios

El módulo de servicios permite al representante activar o desactivar los servicios disponibles para sus diferentes sede mediante toggles (Figura 36). Los servicios se clasifican por tipo.

Figura 36

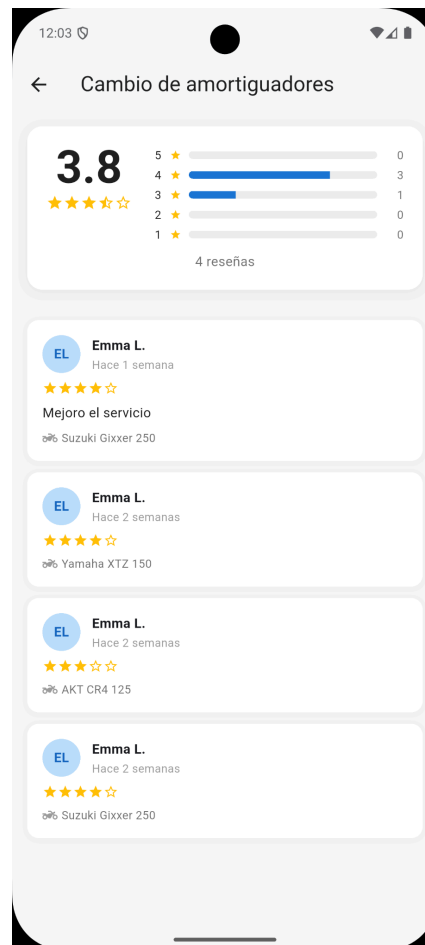
Gestión de servicios con filtros por categoría y toggles de activación.



Cada servicio registrado tiene el detalle de reseñas con la calificación promedio, distribución de estrellas y comentarios individuales de cada usuario que fue atendido, incluido el modelo de motocicleta (Figura 37).

Figura 37

Reseñas de un servicio con distribución de calificaciones por estrellas.



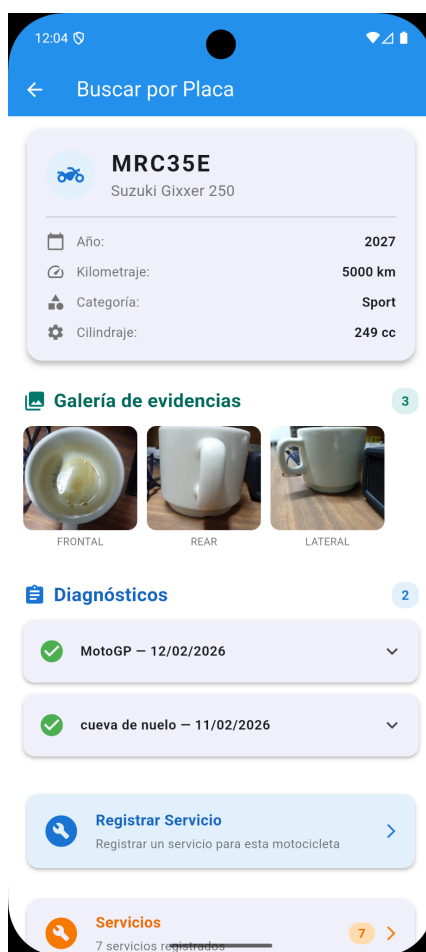
9.3.5 Búsqueda de motocicleta por placa

Una funcionalidad clave para el representante de una sede es la búsqueda de motocicletas por placa. Al ingresar la placa, el sistema muestra toda la información del vehículo: marca, línea, cilindrada, año, kilometraje, galería de evidencia fotográfica y

el historial completo de diagnósticos realizados (Figura 38). Desde aquí, el representante puede registrar un nuevo servicio o ver los servicios existentes.

Figura 38

Búsqueda de motocicleta por placa con información completa, evidencias fotográficas y diagnósticos.



9.3.6 Registro y seguimiento de servicios

el representante registra los servicios a través de un formulario tipo bottom sheet donde selecciona la sede, ingresa la cotización, el precio final y notas del representante (Figura 39).

Figura 39

Formulario de registro de un nuevo servicio con cotización y precio final.

12:05

← Buscar por Placa

MRC35E

Registrar Servicio

Seleccionar Sede

Elige una sede

Primero selecciona una sede para ver los servicios disponibles.

Cotización

Precio Final

Notas del Representante

0/500

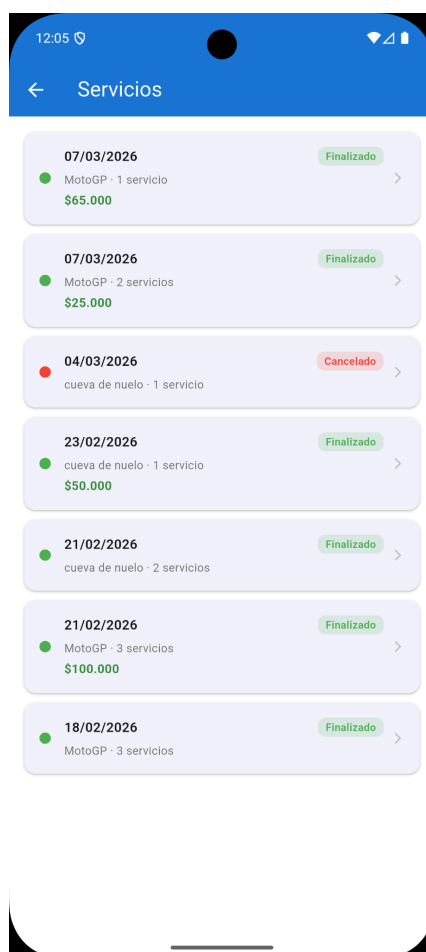
Registrar Servicio

La lista de servicios muestra el historial completo con fecha, sede, número de servicios realizados, precio y estado (Figura 40). Los colores de los indicadores

permiten identificar rápidamente el estado de cada servicio ya sea Completado, Cancelado o Proceso.

Figura 40

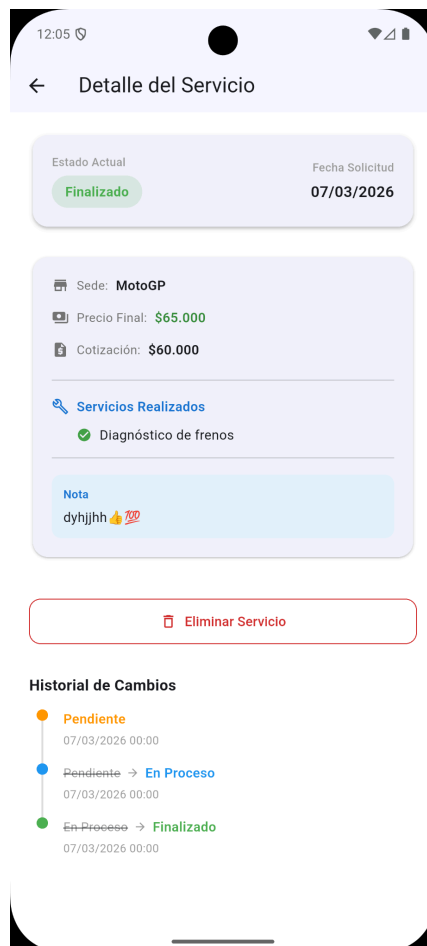
Lista de servicios realizados con estados, precios y fechas.



El detalle de cada servicio muestra: estado actual con código de color, fecha de solicitud, sede asociada, precio, servicios realizados, notas del representante, y un **historial de cambios** tipo timeline que muestra las transiciones de estado con sus fechas. (Figura 41).

Figura 41

Detalle de servicio con historial de cambios tipo timeline.



9.3.7 Gestión de franquicias

El sistema soporta la agrupación de sedes en franquicias. El representante puede crear una franquicia, asignarle un nombre y descripción, y asociar las sedes que le pertenecen (Figura 42). En la Figura 43 se muestra el detalle de una franquicia con las sedes asociadas y las disponibles para agregar.

Figura 42

Formulario de creación de franquicia con asociación de sedes

Figura 43

Detalle de franquicia con sedes asociadas y sedes disponibles.

The image displays two screenshots of an Android application interface for managing franchises and their locations.

Left Screenshot: Crear Franquicia

- Header:** "Crear Franquicia" with a back arrow.
- Form Fields:**
 - Nombre de la franquicia (with a building icon)
 - Descripción (opcional)
- Asociar Sedes:**
 - Subheader: "Asociar Sedes"
 - Text: "Selecciona las sedes que pertenecerán a esta franquicia"
 - Three selectable items:
 - ☐ Hello (Calle 48B #67-90)
 - ☐ cueva de nuelo (Carrera 54B # 37 - 25)
 - ☐ MotoGP (Cra. 46 #40b-50 Sector 3)
- Footer:** "Crear Franquicia" button.

Right Screenshot: Franquicia: nuelo

- Header:** "Franquicia: nuelo" with a back arrow, edit icon, and menu icon.
- Sedes en esta franquicia (1):**
 - Item: **Sucursal MotoMax Norte** (Tienda icon) with address "Cl 44 # 52 - 165, la candelaria, Medellín, La candelaria, Medellín, Antioquia" and a link icon.
- Sedes disponibles (3):**
 - Item: **Hello** (Tienda icon) with address "Calle 48B #67-90" and a "+" icon.
 - Item: **cueva de nuelo** (Tienda icon) with address "Carrera 54B # 37 - 25" and a "+" icon.
 - Item: **MotoGP** (Tienda icon) with address "Cra. 46 #40b-50 Sector 3" and a "+" icon.

9.4 Respuesta a la pregunta de investigación

La pregunta que se planteó fue: ¿Cómo mejorar la gestión en la prestación de servicios informativos y de asistencia técnica para la reparación y el mantenimiento de motocicletas en el municipio de Rionegro, mediante la utilización de una plataforma de software móvil Android?

10. CONCLUSIONES

Ahora bien, con todo lo que se evidencia, queda claro que se proporciona una solución a la pregunta de investigación desde varios frentes:

1. Servicios informativos: Antes, un motociclista tenía que preguntar de boca en boca o buscar en redes sociales para encontrar un taller. Ahora lo que pasa es que con el mapa y los filtros que pusimos de marca y cilindrada, el motociclista abre el celular y ya tiene los talleres ahí, al instante. (Figura 6). Ahí mismo puede ver dónde queda el taller, a qué horas atiende, qué servicios tiene y qué han dicho otros usuarios; sin tener que buscar en otro lado.

2. Asistencia técnica: Con el flujo de diagnóstico que armamos, el motociclista puede mandar fotos de lo que le pasa a la moto y pedir ayuda directo desde la app, que además se conecta con WhatsApp. Así el taller ya tiene toda la información de la moto antes de que el cliente llegue (Figura 38), y eso hace que todo sea más ágil y el servicio quede mejor.

3. Trazabilidad del mantenimiento: El historial de servicios con estados, precios, notas del mecánico y cronograma de cambios le brinda al motociclista

visibilidad total sobre el mantenimiento de cada una de sus motocicletas (Figuras 12 y 39). Esto antes no existía digitalmente en Rionegro.

4. Gestión profesionalizada de los talleres: Los representantes de la sede ahora cuentan con herramientas digitales para administrar sus servicios (Figura 36), horarios (Figura 30), ubicación (Figura 32) y evidencia fotográfica. Esto profesionaliza el funcionamiento de sus establecimientos y les da visibilidad ante los clientes potenciales.

5. Ecosistema conectado: La plataforma Android conecta a los dos principales actores, los motociclistas y representantes de las sedes en un ecosistema digital que cubre todo el ciclo de vida del servicio de motocicletas, desde la búsqueda del taller hasta la calificación del servicio recibido (Figura 14).

Lo primero que se concluye es que el motociclista ya no tiene que buscar un establecimiento a ciegas. Con el mapa interactivo y los filtros por marca, cilindrada y ubicación, la información de los establecimientos está disponible al instante en el celular. Eso ya le ahorra tiempo, contribuye a la reducción de la huella de carbono y sobre todo la frustración de ir de un lugar a otro sin encontrar lo que se necesita.

Lo segundo es la transparencia. Las reseñas y calificaciones que dejan los usuarios dándole confianza a otros motociclistas que necesitan un servicio por primera vez de un establecimiento. tradicionalmente, la única forma de saber si un lugar era bueno era preguntarle a alguien de confianza, y eso no siempre es posible cuando, más aún se está de paso.

El tercer aspecto es la trazabilidad. Ahora hay un historial digital de los servicios que se le han hecho a cada moto, con estados, y notas del mecánico, tener esto de forma digital permite que tanto el motociclista como al establecimiento tener un registro claro de lo que se ha trabajado en una moto.

Para los talleres, MotoGo profesionaliza la forma en que manejan su negocio. Les da herramientas para gestionar sedes, horarios, excepciones, franquicias y servicios de manera organizada. La búsqueda por placa permite tener la información de la moto antes de que el cliente llegue y eso acelera la atención considerablemente.

MotoGo se pensó para ser usado por tres tipos de persona, cada una tiene una experiencia distinta dentro de la aplicación.

El motociclista abre la app y lo primero con lo que se encuentra es el mapa con los establecimientos cerca, luego toca un filtro, por ejemplo Yamaha, o motos de 150

cc y el mapa le deja solo los que le sirven, si se le da en ver detalle, ve los horarios, servicios y las calificaciones, y decide si va a ir o no. también puede registrar sus motos, pedir diagnósticos con fotos del vehículo y ver el historial de lo que le han hecho.

Al representante de la sede, por su lado, la app le permite administrar el negocio sin complicaciones. Puede manejar una o varias sedes, programar horarios y agregar excepciones si algún día no abre, además de activar los servicios que quiera ofrecer o, por el contrario, quitarlos. Si necesita buscar una moto antes de recibirla, lo hace por placa. Todo servicio que registre queda con cotización, precio y notas del mecánico; y si un motociclista le deja reseña, le sirve para saber en qué mejorar.

El role del administrador no interactúa con motos ni talleres directamente. Lo que hace es mantener los catálogos del sistema, es decir las referencias de motos.

Una decisión que se toma desde el diseño es reducir la cantidad de pasos necesarios para completar cualquier acción. Si el motociclista está registrando una moto, las validaciones del formulario aparecen cuando le da enviar y no después de enviar, en cuanto a los servicios realizados usan colores distintos según el estado, amarillo si está en proceso, verde si está finalizado, para que el usuario identifique en qué va.

Las 88 historias de usuario se completaron y la plataforma funciona. Aún así, siempre van a existir oportunidades para madurar la solución, por lo tanto, existe la posibilidad de incluir nuevas funcionalidades dentro del sistema. Por ejemplo:

- Reportes para talleres: hoy el representante no tiene una pantalla donde vea cuántos servicios hizo en el mes, cuál es su calificación promedio o cuánto se demora en promedio. Agregando eso, podría tomar decisiones con números y no con intuición.
- Pagos en línea: actualmente la no hay cotización que llegó por la app. A futuro se podría conectar con una pasarela como Wompi o Bold para que el usuario pague sin cerrar MotoGo.
- Soporte para iOS: Dado que el frontend está construido en Flutter, la app puede compilar nativamente para iOS. Expandir la plataforma al ecosistema Apple ampliaría significativamente la base de usuarios.

Estas mejoras no son necesarias para que el producto funcione, pero le darían mayor profundidad. La arquitectura con la que se construyó MotoGo está preparada para recibir estos módulos sin necesidad de reestructurar lo que ya existe.

Algunas de las dificultades que se presentaron, en su mayoría, cada vez que era necesario integrar servicios externos al negocio, se generaban cuellos de botella que dificultaban la continuidad del desarrollo.

Finalmente este trabajo fue un constante aprendizaje desde la planeación y el análisis del problema hasta su implementación, de manera ordenada, consolidando los conocimientos adquiridos en las aulas.

11. REFERENCIAS BIBLIOGRÁFICAS

- 1 .Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.
2. Brandolini, A. (2021). Introducing EventStorming: An Act of Deliberate Collective Learning. Leanpub.
https://leanpub.com/introducing_eventstorming
3. Patton, J. (2014). User Story Mapping: Discover the Whole Story, Build the Right Product. O'Reilly Media.
4. Wynne, M., & Hellesøy, A. (2017). The Cucumber Book: Behaviour-Driven Development for Testers and Developers (2nd ed.). Pragmatic Bookshelf.
5. Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley Professional.
6. Adzic, G. (2012). Impact Mapping: Making a Big Impact with Software Products and Projects. Provoking Thoughts.
7. Parasuraman, A., Zeithaml, V. A., & Berry, L. L. (1988). SERVQUAL: A Multiple-Item Scale for Measuring Consumer Perceptions of Service Quality. *Journal of Retailing*, 64(1), 12–40.
8. Norman, D. (2013). The Design of Everyday Things (Revised and Expanded Edition). Basic Books.

9. Mayer, R. C., Davis, J. H., & Schoorman, F. D. (1995). An Integrative Model of Organizational Trust. *Academy of Management Review*, 20(3), 709–734.
10. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
11. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
12. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
13. Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
14. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
15. AutoSoft Taller. (s.f.). Software para administración de talleres automotrices. <https://autosofttaller.com>
16. El Colombiano. (27 de febrero de 2026). Parque automotor en Colombia superó los 21,3 millones de vehículos en 2025: motos lideran y crecen eléctricos e híbridos. <https://www.elcolombiano.com/negocios/parque-automotor-colombia-2025-NP33968207>
17. Google LLC. (s.f.). Google Maps. <https://maps.google.com>
18. La Gran Noticia. (27 de febrero de 2026). El parque automotor de Colombia: 21.345.925 vehículos activos en el Runt. <https://lagrannoticia.com/el-parque-automotor-de-colombia-21-345-925-vehiculos-activos-en-el-runt/>
19. Registro Único Nacional de Tránsito [RUNT]. (2025). Boletín de prensa 001 de 2025. <https://www.runt.gov.co/sites/default/files/Bolet%C3%ADn%20de%20Prensa%20001%20de%202025.pdf>

20. ServitechApp. (s.f.). Software de gestión para talleres.
<https://servitechapp.com>
21. TuneraSoft. (s.f.). TuneraTaller: Software de gestión para talleres en la nube. <https://programa-taller.com>
22. Yelp Inc. (s.f.). Yelp: Find local businesses, reviews, and more.
<https://www.yelp.com>
23. Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., & Diakopoulos, N. (2016). Designing the user interface: Strategies for effective human–computer interaction (6th ed.). Pearson.
24. Goodchild, M. F. (2007). Citizens as sensors: The world of volunteered geography. *GeoJournal*, 69(4), 211–221.
<https://doi.org/10.1007/s10708-007-9111-y>

12. ANEXOS

Se listan los materiales de soporte que complementan lo expuesto a lo largo del documento. El anexo se organiza con base en la fase del diseño metodológico a la que pertenece y se identifica con un número consecutivo.

hace referencia a ”<Repositorio institucional>” como la ubicación donde oficialmente la universidad ha dispuesto en custodia los artefactos, fruto de este trabajo de grado de acuerdo a lo especificado en cada uno de los anexos.

Tabla 2*Anexos de artefactos estratégicos del proyecto MotoGo*

Nº	Nombre	Ruta
1	Visión	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\1. Visión
2	Impact Mapping	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\2. Mapa de Impacto
3	Modelo de dominio	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\3. Modelo de dominio\Requisitos de información
4	Event Storming	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\4. Event Storming
5	Mapa de historias de usuario	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\5. Mapa de historias de usuario
6	Requerimientos Funcionales	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\6. Especificación funcional\3. Requerimientos\Funcionales
7	Requerimientos No Funcionales	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\6. Especificación funcional\3. Requerimientos\No funcionales
8	Reglas de Negocio	<Repositorio institucional>\motogo\1.Artefactos proyecto\1. Artefactos estratégicos\6. Especificación funcional\3. Requerimientos\Reglas de negocio

Tabla 3*Anexos de artefactos técnicos del proyecto MotoGo*

Nº	Nombre	Ruta
9	Atributos de calidad	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\1. Atributos de calidad
10	Funcionalidades críticas	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\2. Funcionalidades criticas
11	Restricciones técnicas	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\3. Restricciones técnicas

12	Restricciones de negocio	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\1. Drivers arquitectónicos\4. Restricciones de negocio
13	Spikes (Pruebas de concepto)	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\2. Spikes (Pruebas de concepto)
14	Alternativas de solución	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\3. Alternativas de solución
15	Arquetipo de solución	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\4. Arquetipo de solución
16	Stack tecnológico	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\1. Diseño Arquitectónico\5. Plataforma tecnológica
17	Diagrama de clases	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\1. Diagrama de clases
18	Diagrama de componentes	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\2. Diagrama de componentes
19	Diagrama de paquetes	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\3. Diagrama de paquetes
20	Diagramas de secuencia	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\4. Diagrama de secuencia
21	Diagrama de despliegue	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\5. Diagrama de despliegue
22	Modelo Entidad Relación	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\6. Modelo entidad relación
23	Diagrama de Actividades	<Repositorio institucional>\motogo\1.Artefactos proyecto\2. Artefactos técnicos\2. Diseño detallado\8. Diagrama de Actividades

Tabla 4*Anexos de construcción del proyecto MotoGo*

Nº	Nombre	Ruta
24	Análisis estático de código	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\1. Calidad de código\Análisis_Estatico_MotoGo.xlsx

25	Informe de pruebas de carga	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\1. Calidad de código\Informe_Test_Carga_MotoGo.xlsx
26	Código fuente API	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\2. Código fuente\api
27	Configuración Backend	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\2. Código fuente\api\backend_Configuración_MotoGo
28	Código fuente App	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\2. Código fuente\app
29	APK MotoGo	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\2. Código fuente\app\Apk_MotoGo\MotoGo.apk
30	CI/CD Pipeline (Devops)	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\3. Devops\CICD_Pipeline_MotoGo.xlsx
31	Patrones de Diseño	<Repositorio institucional>\motogo\1.Artefactos proyecto\3.Construcción\4. Patrones de Diseño\Patrones_Diseno_MotoGo.xlsx

Tabla 5*Anexos de puesta en marcha del proyecto MotoGo*

Nº	Nombre	Ruta
32	Manual de instalación de bases de datos	<Repositorio institucional>\motogo\1.Artefactos proyecto\4. Puesta en marcha\Manual_Instalacion_BD_MotoGo.xlsx
33	Manual de instalación	<Repositorio institucional>\motogo\1.Artefactos proyecto\4. Puesta en marcha\Manual_Instalacion_MotoGo.xlsx
34	Manual de despliegue	<Repositorio institucional>\motogo\1.Artefactos proyecto\4. Puesta en marcha\Manual_Despliegue_MotoGo.xlsx
35	Dump base de datos de negocio	<Repositorio institucional>\motogo\1.Artefactos proyecto\4. Puesta en marcha\dump_motogo.sql
36	Dump base de datos Keycloak	<Repositorio institucional>\motogo\1.Artefactos proyecto\4. Puesta en marcha\dump_keycloak.sql
37	Schema base de datos de negocio	<Repositorio institucional>\motogo\1.Artefactos proyecto\4. Puesta en marcha\schema-motogo.sql
38	Schema base de datos Keycloak	<Repositorio institucional>\motogo\1.Artefactos proyecto\4. Puesta en marcha\schema-keycloak.sql

Tabla 6*Abreviaturas utilizadas en el proyecto MotoGo*

Abreviatura	Significado
API	Application Programming Interface
APK	Android Package Kit
BDD	Behavior Driven Development
BLoC	Business Logic Component
CD	Continuous Deployment
CI	Continuous Integration
CI/CD	Continuous Integration / Continuous Deployment
DDD	Domain-Driven Design
DNS	Domain Name System
DTO	Data Transfer Object
GB	Gigabyte
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
HU	Historia de Usuario
JSON	JavaScript Object Notation
JWT	JSON Web Token
MB	Megabyte
OAuth2	Open Authorization 2.0
PKCE	Proof Key for Code Exchange
RAM	Random Access Memory
REST	Representational State Transfer
RUNT	Registro Único Nacional de Tránsito
SEI	Software Engineering Institute
SERVQUAL	Service Quality

SQL	Structured Query Language
SSH	Secure Shell
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UUID	Universally Unique Identifier
UX	User Experience
VPS	Virtual Private Server

13. GLOSARIO

A continuación se presentan las definiciones de los términos técnicos empleados en el presente documento, con el fin de facilitar la comprensión por parte de lectores de diversas áreas del conocimiento.

API: Son las reglas que le permiten a dos aplicaciones hablar entre sí. En MotoGo la API es lo que conecta la app del celular con el servidor.

API REST: Es un estilo de diseño para construir APIs que se apoya en el protocolo HTTP. Se basa en operaciones como consultar, crear, actualizar y eliminar recursos.

Arquitectura Hexagonal: Manera de organizar el código donde la lógica del negocio queda aislada del mundo exterior; o sea, de las bases de datos, las pantallas y cualquier servicio externo. Eso hace que el sistema sea más fácil de probar.

Arquitectura Limpia: Propuesta de Robert C. Martin para separar el software en capas. Las capas de adentro guardan las reglas del negocio y las de afuera se encargan de los detalles técnicos como la base de datos o la interfaz.

Atributos de Calidad: Son las características que definen qué tan bien se porta un sistema más allá de lo funcional: la disponibilidad, la seguridad, el rendimiento y la mantenibilidad entre otros.

Backend: La parte del sistema que vive en el servidor. Procesa las peticiones que llegan desde la app, ejecuta la lógica del negocio y habla con la base de datos.

BDD: Sigla de Behaviour-Driven Development. Es una forma de describir lo que el software debe hacer usando un lenguaje natural estructurado: Dado que... Cuando... Entonces.

BLoC: Sigla de Business Logic Component. Es un patrón que se usa en Flutter para separar la lógica del negocio de lo visual, lo cual facilita mantener y probar el código.

Bounded Context: Viene del Diseño Orientado al Dominio. Es la frontera que delimita hasta dónde aplica un modelo particular, evitando que los conceptos se mezclen con los de otro módulo.

CI/CD: Integración Continua y Despliegue Continuo. Automatiza la compilación, las pruebas y la puesta en producción del software cada vez que se hacen cambios en el código.

Contenedor Docker: Empaqueta todo lo que necesita una aplicación para correr: el código, las librerías y la configuración. Gracias a eso se comporta igual sin importar en qué máquina lo pongas.

DDD: Sigla de Domain-Driven Design o Diseño Orientado al Dominio. Lo propuso Eric Evans y consiste en centrar el desarrollo del software en entender bien las reglas y procesos del negocio real.

Despliegue: Es el proceso de subir la aplicación a un servidor para que los usuarios la puedan usar por internet. También se le conoce como deploy.

Docker Compose: Herramienta que coordina varios contenedores Docker al tiempo. Sirve para levantar de un solo golpe la base de datos, el servidor y los servicios de monitoreo.

Dockerfile: Archivo que contiene las instrucciones paso a paso para construir la imagen de un contenedor: qué sistema operativo base usar, qué instalar y cómo arrancar la aplicación.

Dokploy: Plataforma autohospedada que facilita la gestión de despliegues. Funciona como un PaaS propio y evita tener que entrar manualmente al servidor cada vez que hay un cambio.

Driver Arquitectónico: Cualquier factor que influye en cómo se diseña la arquitectura del software. Puede ser un requisito de seguridad, una limitación de rendimiento o una restricción tecnológica.

DTO: Data Transfer Object. Es un objeto cuyo único trabajo es transportar datos entre las capas del sistema; no tiene lógica de negocio adentro.

Endpoint: Dirección específica dentro de una API. Por ejemplo, /motorcycles es el endpoint donde se gestionan las motocicletas del sistema.

Entidad de Dominio: Un objeto del software que representa algo del mundo real del negocio: una motocicleta, una sede o un diagnóstico. Cada entidad tiene identidad propia y reglas que la gobiernan.

Event Storming: Técnica de descubrimiento de procesos creada por Alberto Brandolini. Se identifican todos los eventos del negocio y se ponen en orden cronológico para entender el flujo real.

Firestore: Plataforma de Google con servicios listos para apps móviles: almacenamiento de archivos, autenticación de usuarios y envío de notificaciones push entre otros.

Flutter: Framework de Google para crear aplicaciones móviles, web y de escritorio usando una sola base de código escrita en el lenguaje Dart.

Framework: Conjunto de herramientas y convenciones que simplifican el desarrollo de software. Le da al programador una base sobre la cual construir sin arrancar de cero.

Frontend: Todo lo que el usuario ve y toca en la aplicación: pantallas, botones, formularios y la navegación entre ellos.

Geolocalización: Capacidad que tiene el sistema para saber dónde está el usuario en tiempo real, usando el GPS del celular u otros sensores de ubicación.

Gin: Framework web para el lenguaje Go. Se usa bastante para construir APIs que necesitan responder rápido y manejar muchas peticiones al tiempo.

Go: Lenguaje de programación desarrollado por Google. Es eficiente y simple; se usa mucho para servidores y servicios que necesitan alto rendimiento. También se le conoce como Golang.

Grafana: Herramienta que muestra en paneles visuales las métricas del sistema. Se usa para ver en tiempo real cómo se está comportando el servidor, la base de datos y los servicios.

Handler: En el backend es el componente que recibe la petición HTTP del usuario, la procesa y le devuelve una respuesta. Es la puerta de entrada al sistema.

Historia de Usuario: Descripción corta de una funcionalidad vista desde quien la va a usar. Se redacta así: Como tal rol, quiero tal cosa, para obtener tal beneficio. Se abrevia HU.

HTTP / HTTPS: Protocolo que usan los navegadores y las apps para comunicarse con un servidor. HTTPS es la versión cifrada que protege los datos en tránsito.

Impact Mapping: Técnica de planificación que propuso Gojko Adzic. Conecta el objetivo del proyecto con los actores, los impactos que se quieren lograr y lo que hay que construir para conseguirlo.

Interactor: Componente que orquesta la lógica de un caso de uso. Coordina la comunicación entre los servicios y los repositorios sin estar amarrado a detalles de la interfaz o la base de datos.

Inyección de Dependencias: Técnica donde cada componente recibe lo que necesita desde afuera en vez de crearlo por dentro. Eso hace que sea más sencillo reemplazar piezas del sistema o hacer pruebas.

JSON: Formato liviano para intercambiar datos entre la app y el servidor. Se lee fácil tanto por humanos como por máquinas. Su nombre completo es JavaScript Object Notation.

JWT: Sigla de JSON Web Token. Es un estándar que permite verificar quién es el usuario de forma segura, enviando datos firmados digitalmente entre la app y el servidor.

k6: Herramienta de Grafana para hacer pruebas de carga. Simula muchos usuarios al mismo tiempo para ver cómo se comporta el servidor bajo presión.

Keycloak: Servidor de gestión de identidad de código abierto. Se encarga del registro, el inicio de sesión y el control de permisos de los usuarios.

Lenguaje Ubicuo: Vocabulario común entre el equipo técnico y la gente del negocio. Lo propuso Eric Evans en DDD para que todos hablen de lo mismo sin ambigüedades. En inglés se conoce como Ubiquitous Language.

Let's Encrypt: Entidad que emite certificados SSL de forma gratuita y automática para que los sitios web puedan cifrar la comunicación con los usuarios.

Loki: Sistema de Grafana Labs que centraliza y almacena los logs de las aplicaciones. Permite buscar y consultar los registros de eventos de todos los contenedores en un solo lugar.

Middleware: Pieza de software que se ejecuta antes de que la petición llegue al destino final. Ahí se validan tokens, se registran logs o se verifican permisos.

Modelo de Dominio: Representación de los objetos, las reglas y las relaciones del negocio. Es como el mapa que le dice al código cómo funciona el mundo real que está modelando.

MySQL: Motor de base de datos relacional de código abierto. Almacena los datos del sistema en tablas organizadas con filas y columnas.

Máquina de Estados: Modelo que define los estados por los que puede pasar un objeto y las transiciones válidas entre ellos. Un diagnóstico por ejemplo arranca en SOLICITADO, pasa a EN_PROCESO y termina en COMPLETADO.

OAuth2 / PKCE: OAuth2 es un protocolo que permite a la app acceder a recursos del usuario sin necesitar su contraseña directamente. PKCE es una capa extra de seguridad pensada para aplicaciones móviles.

Patrón de Diseño: Solución ya probada para un problema que aparece seguido en el desarrollo de software. Algunos ejemplos son Repository, Singleton y Strategy.

Pipeline: Secuencia de pasos automatizados que compilan, prueban y despliegan el software de manera ordenada cada vez que se hace un cambio.

Product Backlog: Lista que reúne todas las funcionalidades y mejoras pendientes del producto, ordenadas por la prioridad que le da el negocio.

Prometheus: Sistema de monitoreo que recopila métricas del servidor en forma de números: uso de memoria, tiempos de respuesta, cantidad de peticiones y demás indicadores de salud del sistema.

Proxy Inverso: Servidor que recibe las peticiones de los usuarios y decide a cuál servicio interno redirigirlas. En MotoGo ese papel lo cumple Traefik.

Quality Gate: Conjunto de condiciones mínimas que el código debe cumplir para ser aceptado. Por ejemplo: cero bugs críticos, cobertura de pruebas por encima de cierto porcentaje y duplicación de código controlada.

Repository Pattern: Patrón que pone una capa intermedia entre la lógica del negocio y la base de datos. Le da al resto del código una interfaz limpia para guardar, consultar o borrar registros sin mezclar detalles de SQL.

SonarQube: Plataforma que analiza el código fuente sin ejecutarlo. Detecta errores, posibles vulnerabilidades y mide qué tan cubierto está el código por pruebas automatizadas.

Spike: Investigación técnica corta que se hace cuando hay incertidumbre sobre una tecnología o un enfoque. El objetivo es aprender y decidir antes de ponerse a implementar.

SSL/TLS: Protocolos que cifran la comunicación entre el navegador del usuario y el servidor. Garantizan que nadie pueda leer los datos mientras viajan por la red.

Struct: En Go es la estructura que agrupa varios campos bajo un mismo tipo. Vendría siendo el equivalente a una clase en lenguajes como Java o C#.

Test Unitario: Prueba automatizada que verifica que una sola pieza del código funcione bien de forma aislada, sin depender de bases de datos ni servicios externos.

Throughput: Cantidad de peticiones que el servidor logra procesar en un segundo. Es una de las métricas clave para medir el rendimiento bajo carga.

Traefik: Proxy inverso moderno que se integra con Docker. Dirige el tráfico web al contenedor correcto según el dominio y gestiona los certificados SSL automáticamente con Let's Encrypt.

User Story Mapping: Técnica de Jeff Patton para visualizar todas las historias de usuario en un mapa de dos ejes: las actividades del usuario en la horizontal y las versiones del producto en la vertical.

UUID: Identificador único universal de 128 bits. Se usa para que cada registro en la base de datos tenga un ID irrepetible sin depender de una secuencia numérica.

VPS: Servidor virtual privado alojado en la nube. Tiene procesador, memoria y disco dedicados para correr las aplicaciones en producción.

Webhook: Notificación automática que un servicio le manda a otro cuando pasa algo.

En MotoGo, GitHub le avisa a Dokploy mediante un webhook cada vez que hay cambios en el código para que arranque el despliegue.